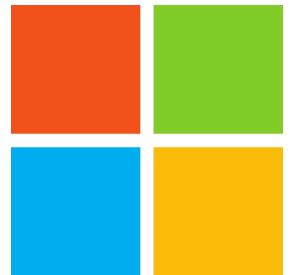


Understanding Understanding (in Coding Agents)

Presented by [Nishant](#) :)



Programming today is really different

Task: Fix the return value in *merch.py*

```
merch.py
1 def get_best_merch(idx):
2     merch = [
3         "Microsoft",
4         "Ai2",
5     ]
6     return merch[idx]
```



The best company merch is missing!!!

Programming today is really different

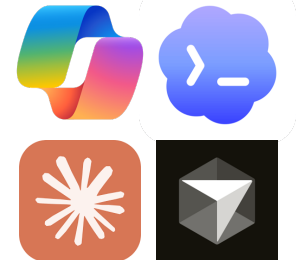
Task: Fix the return value in *merch.py*

```
merch.py
1 def get_best_merch(idx):
2     merch = [
3         "Microsoft",
4         "Ai2",
5     ]
6     return merch[idx]
```

Read code, plan solution, execute, ...



Agents



Hey Clopus, please go fix this list

Programming today is really different

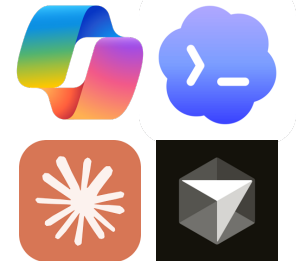
Task: Fix the return value in *merch.py*

```
merch.py
1 def get_best_merch(idx):
2     merch = [
3         # fixed list
4         "Pangram",
5         "Microsoft",
6         "Ai2",
7     ]
8     return merch[idx]
9     return merch[0]
```

Read code, plan solution, execute, ...



Agents



Hey Clopus, please go fix this list

Programming today is really different

Task: Fix the return value in *merch.py*

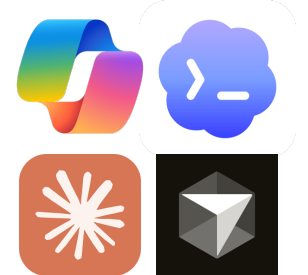
```
merch.py
1 def get_best_merch(idx):
2     merch = [
3         # fixed list
4         "Pangram",
5         "Microsoft",
6         "Ai2",
7     ]
8     return merch[idx]
9     + return merch[0]
```

Pangram has usurped the throne!



Hey Clopus, please go fix this list

Agents



Programming today is really different

Task: Fix the return value in *merch.py*

```
merch.py
1 def get_best_merch(idx):
2     merch = [
3         # fixed list
4         "Pangram",
5         "Microsoft",
6         "Ai2",
7     ]
8     return merch[idx]
9     return merch[0]
```

Test Cases: Verify the code's accuracy

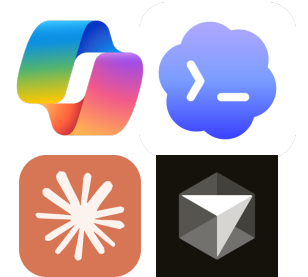
```
assert "Pangram" == get_best_merch(0) ✓
assert "Pangram" == get_best_merch(1) ✓
```

Pangram has usurped the throne!



Hey Clopus, please go fix this list

Agents



Programming today is really different

Task: Fix the return value in *merch.py*

```
merch.py
1 def get_best_merch(idx):
2     merch = [
3         # fixed list
4         "Pangram",
5         "Microsoft",
6         "Ai2",
7     ]
8     return merch[0]
```

Test Cases: Verify the code's accuracy

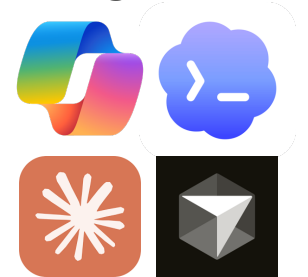
```
assert "Pangram" == get_best_merch(0) ✓
assert "Pangram" == get_best_merch(1) ✓
```

Pangram has usurped the throne!



LGTM!

Agents



And this is how we build programming benchmarks!

Task: Fix the return value in *merch.py*

```
merch.py
1 def get_best_merch(idx):
2     merch = [
3         # fixed list
4         "Pangram",
5         "Microsoft",
6         "Ai2",
7     ]
8     return merch[0]
```

Test Cases: Verify the code's accuracy

```
assert "Pangram" == get_best_merch(0)
```



```
assert "Pangram" == get_best_merch(1)
```



And this is how we build programming benchmarks!

Task: Fix the return value in *merch.py*

```
merch.py
1 def get_best_merch(idx):
2     merch = [
3         # fixed list
4         "Pangram",
5         "Microsoft",
6         "Ai2",
7     ]
8     return merch[0]
```

Test Cases: Verify the code's accuracy

```
assert "Pangram" == get_best_merch(0) ✓
assert "Pangram" == get_best_merch(1) ✓
```

...

Task: Fix the return value in *merch.py*

```
merch.py
1 def get_best_merch(idx):
2     merch = [
3         # fixed list
4         "Pangram",
5         "Microsoft",
6         "Ai2",
7     ]
8     return merch[0]
```

Test Cases: Verify the code's accuracy

```
assert "Pangram" == get_best_merch(0) ✓
assert "Pangram" == get_best_merch(1) ✓
```

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE REAL-WORLD GITHUB ISSUES?

scale

SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?



Test Case Accuracy:
73%

scale

SWE Atlas: Benchmarking Coding Agents Beyond Issue Resolution

REFACTORBENCH: EVALUATING STATEFUL REASONING IN LANGUAGE AGENTS THROUGH CODE

To make “progress”, we get agents to edit more and more code...

GPT-5x Accuracy: 0.73
Average # File Edits: 1.2

GPT-5x Accuracy: 0.59
Average # File Edits: 5.1

GPT-5x Accuracy: 0.38
Average # File Edits: 11.0

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE
REAL-WORLD GITHUB ISSUES?

scale
SWE-Bench Pro: Can AI Agents Solve
Long-Horizon Software Engineering Tasks?

scale
SWE Atlas: Benchmarking Coding Agents
Beyond Issue Resolution

*“Solutions involve roughly **2x the lines of code**
changed and **1.7x the file edits** of SWE-Bench Pro”*

To make “progress”, we get agents to edit more and more code...

GPT-5x Accuracy: 0.73
Average # File Edits: 1.2

GPT-5x Accuracy: 0.59
Average # File Edits: 5.1

GPT-5x Accuracy: 0.38
Average # File Edits: 11.0

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE
REAL-WORLD GITHUB ISSUES?

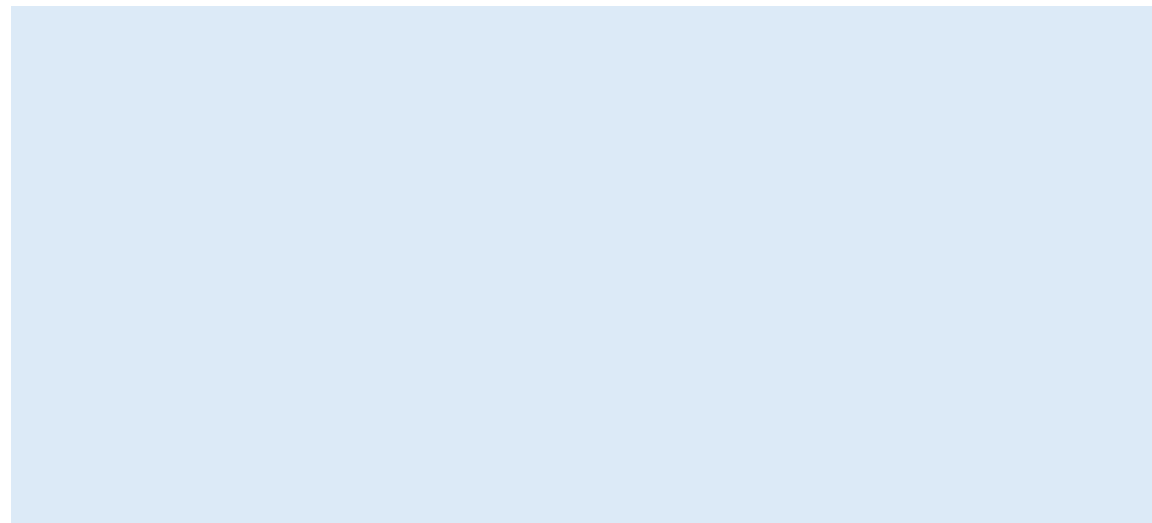
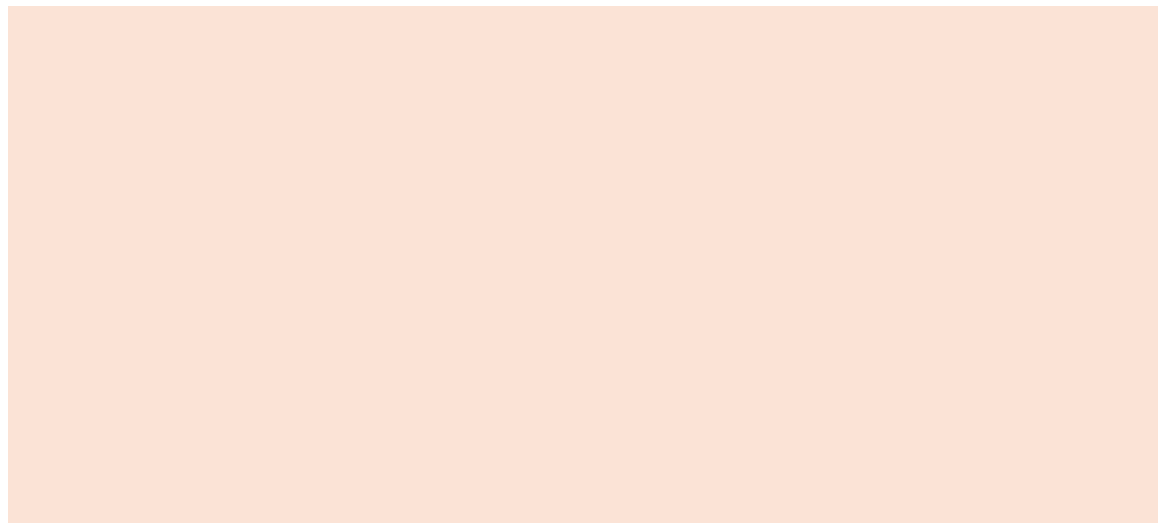
scale

SWE-Bench Pro: Can AI Agents Solve
Long-Horizon Software Engineering Tasks?

scale

SWE Atlas: Benchmarking Coding Agents
Beyond Issue Resolution

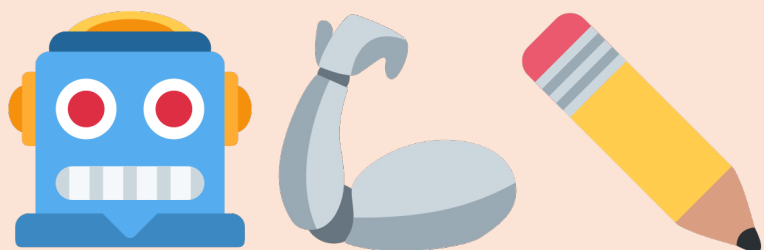
But this motivates two questions for offline + online evals:



To make “progress”, we get agents to edit more and more code...

But this motivates two questions for offline + online evals:

RQ1: Which abilities beyond editing do coding agents need, but lack?



[Programming is Not Just Editing! The Importance of Code Understanding for Agents \(Under Review\)](#)

RQ2: Does optimizing agents for editing support human developers?



[\(Im\)Paired Programming: Coding Agents Improve Productivity but Harm Understanding \(Under Review\)](#)



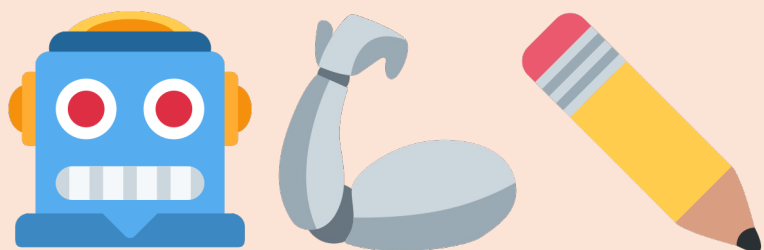
The common link? Understanding is a weakness!



To make “progress”, we get agents to edit more and more code...

But this motivates two questions for **offline** + online evals:

RQ1: Which abilities beyond editing do coding agents need, but lack?



Programming is Not Just Editing! The Importance of Code Understanding for Agents (Under Review)

RQ2: Does optimizing agents for editing support human developers?



(Im)Paired Programming: Coding Agents Improve Productivity but Harm Understanding (Under Review)



The common link? Understanding is a weakness! 🧠

Motivation: Which tasks (beyond editing) are truly difficult for coding agents?

- Discovering agent weaknesses can inform benchmarks, modeling, + deployment
- Repository-based benchmarks are very noisy (e.g., repositories change per evaluation item)
- Synthetic, controlled tasks to the rescue! ([Needle in a Haystack](#)) → unstudied in coding agents
- We introduce **CABRA**¹ to generate code editing tasks from scratch 🐐

[1] Don't worry about the backronym, it's named in honor of our favorite coffee place (La Cabra) near MSR NYC :)

A controlled, three-step agent evaluation framework

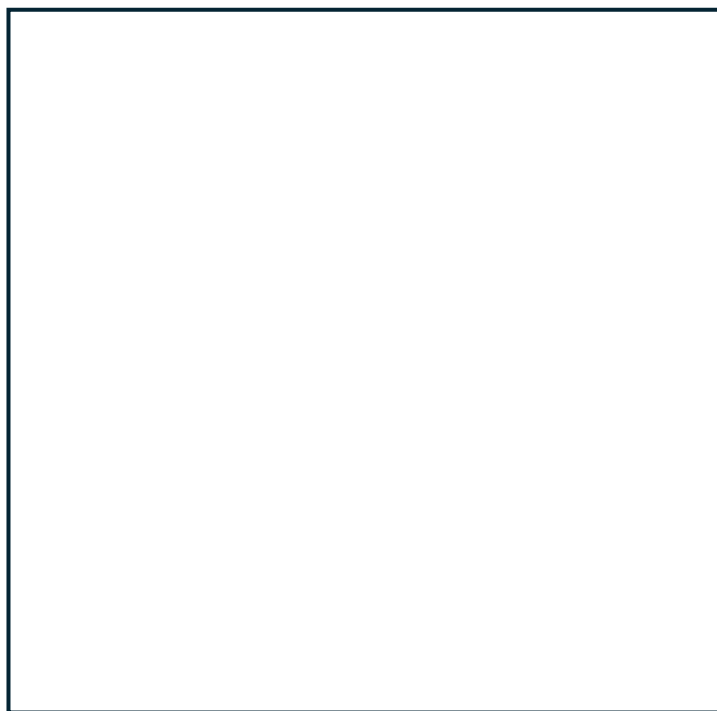
- We introduce **CABRA**¹ to generate code editing tasks from scratch 

A controlled, three-step agent evaluation framework

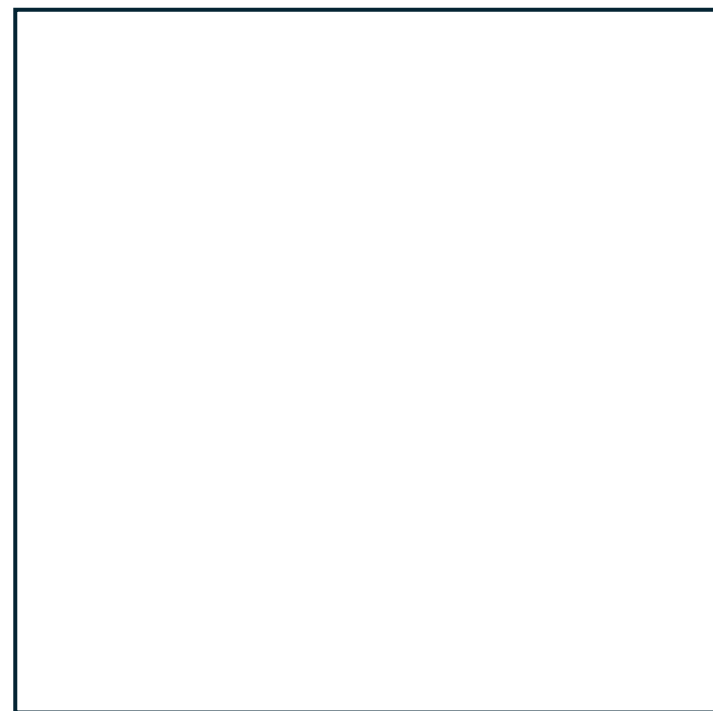
- We introduce **CABRA**¹ to generate code editing tasks from scratch 🐐

Step 1: Turn directed acyclic graphs into programs

Directed Acyclic Graph



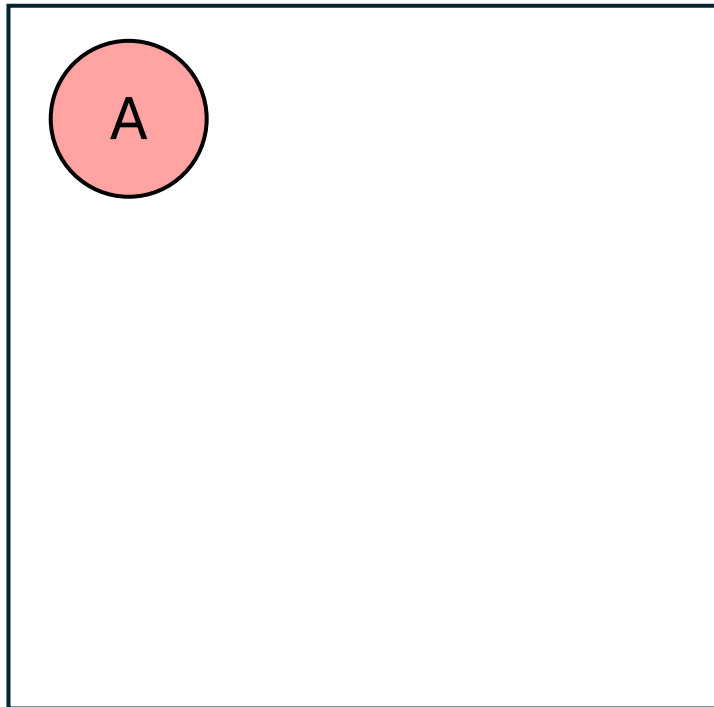
Generated Program



A controlled, three-step agent evaluation framework

- We introduce **CABRA**¹ to generate code editing tasks from scratch 🐐
Step 1: Turn directed acyclic graphs into programs

Directed Acyclic Graph



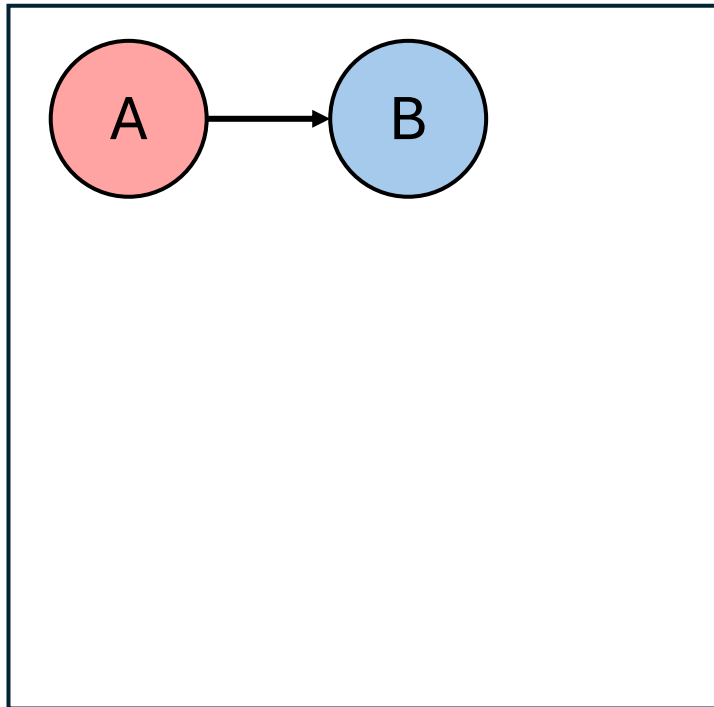
Generated Program

```
def A(val: float) -> float:  
    return ...
```

A controlled, three-step agent evaluation framework

- We introduce **CABRA**¹ to generate code editing tasks from scratch 🐐
Step 1: Turn directed acyclic graphs into programs

Directed Acyclic Graph



Generated Program

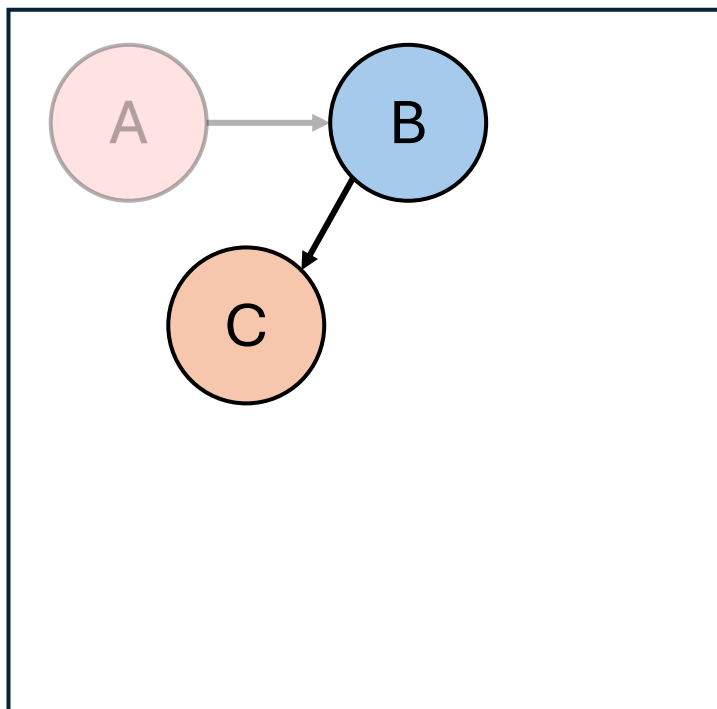
```
def A(val: float) -> float:  
    return math.sin(B(val))  
def B(val: float) -> float:  
    return ...
```

A controlled, three-step agent evaluation framework

- We introduce **CABRA**¹ to generate code editing tasks from scratch 🐐

Step 1: Turn directed acyclic graphs into programs

Directed Acyclic Graph



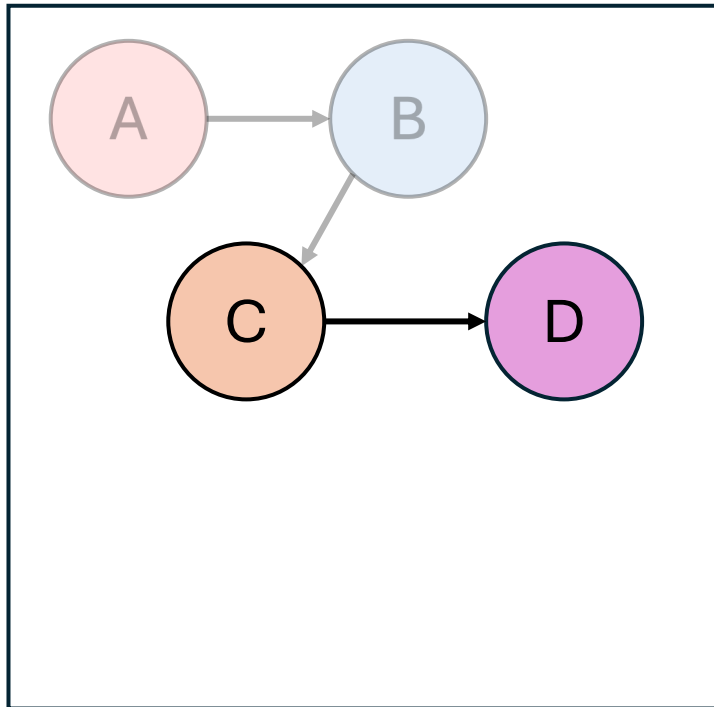
Generated Program

```
def A(val: float) -> float:  
    return math.sin(B(val))  
  
def B(val: float) -> float:  
    return C(val)  
  
def C(val: float) -> float:  
    return ...
```


A controlled, three-step agent evaluation framework

- We introduce **CABRA**¹ to generate code editing tasks from scratch 🐐
Step 1: Turn directed acyclic graphs into programs

Directed Acyclic Graph



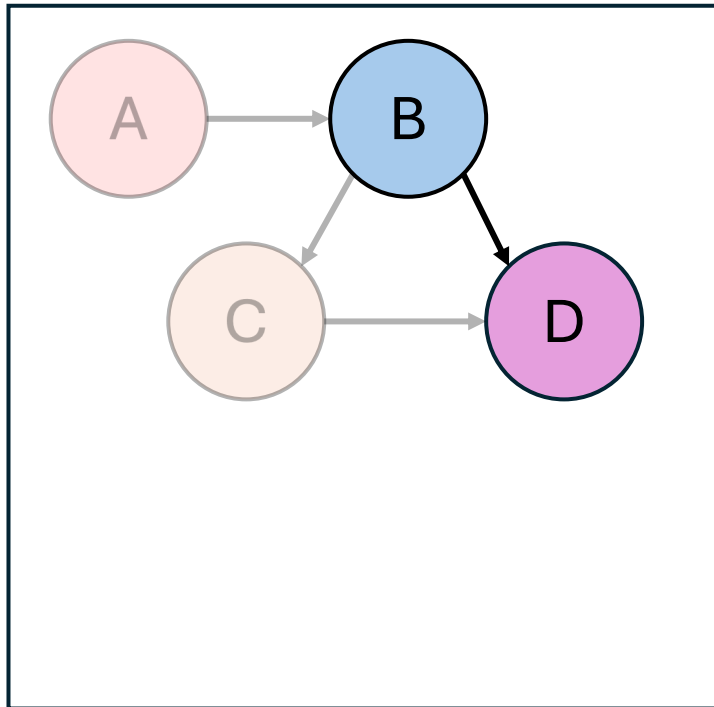
Generated Program

```
def A(val: float) -> float:  
    return math.sin(B(val))  
def B(val: float) -> float:  
    return C(val)  
def C(val: float) -> float:  
    return -1.0 * D(val)  
def D(val: float) -> float:  
    return ...
```

A controlled, three-step agent evaluation framework

- We introduce **CABRA**¹ to generate code editing tasks from scratch 🐐
Step 1: Turn directed acyclic graphs into programs

Directed Acyclic Graph



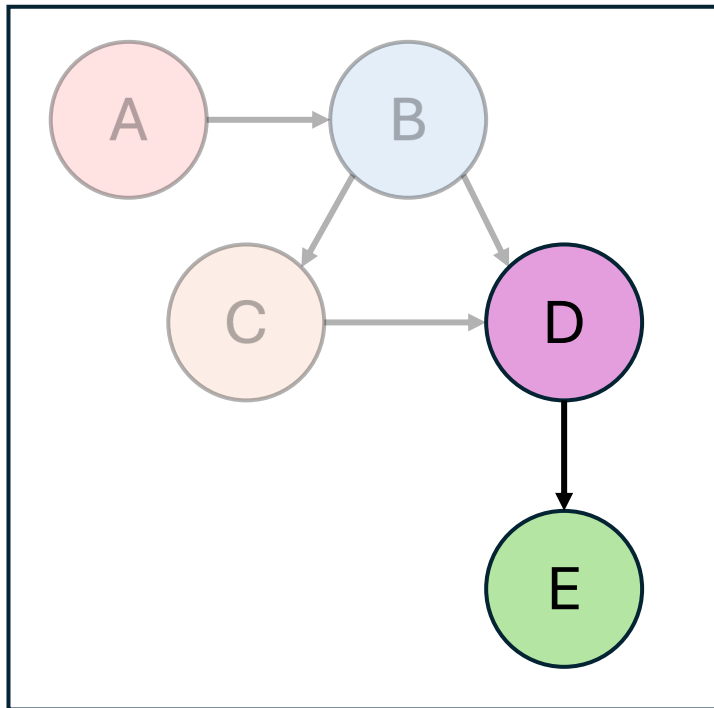
Generated Program

```
def A(val: float) -> float:  
    return math.sin(B(val))  
def B(val: float) -> float:  
    return C(val) * D(val)  
def C(val: float) -> float:  
    return -1.0 * D(val)  
def D(val: float) -> float:  
    return ...
```

A controlled, three-step agent evaluation framework

- We introduce **CABRA**¹ to generate code editing tasks from scratch 🐐
Step 1: Turn directed acyclic graphs into programs

Directed Acyclic Graph



Generated Program

```
def A(val: float) -> float:
    return math.sin(B(val))
def B(val: float) -> float:
    return C(val) * D(val)
def C(val: float) -> float:
    return -1.0 * D(val)
def D(val: float) -> float:
    return 0.5 + E(val)
def E(val: float) -> float:
    return math.sqrt(val)
```

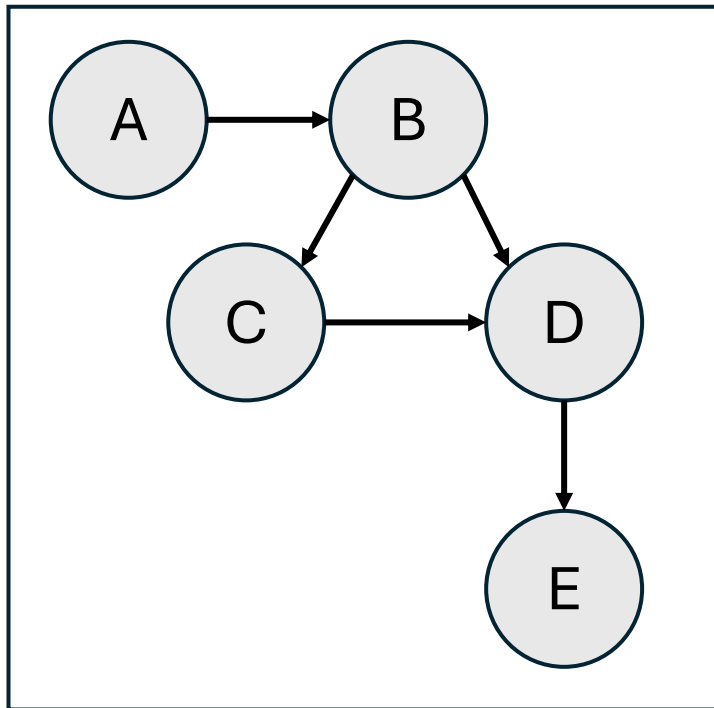
A controlled, three-step agent evaluation framework

- We introduce **CABRA**¹ to generate code editing tasks from scratch 🐐

Step 1: Turn directed acyclic graphs into programs

Step 2: Transform them into five tasks [[Fowler's Refactors](#)]

Directed Acyclic Graph



Generated Program

```
def A(val: float) -> float:
    return math.sin(B(val))

def B(val: float) -> float:
    return C(val) * D(val)

def C(val: float) -> float:
    return -1.0 * D(val)

def D(val: float) -> float:
    return 0.5 + E(val)

def E(val: float) -> float:
    return math.sqrt(val)
```

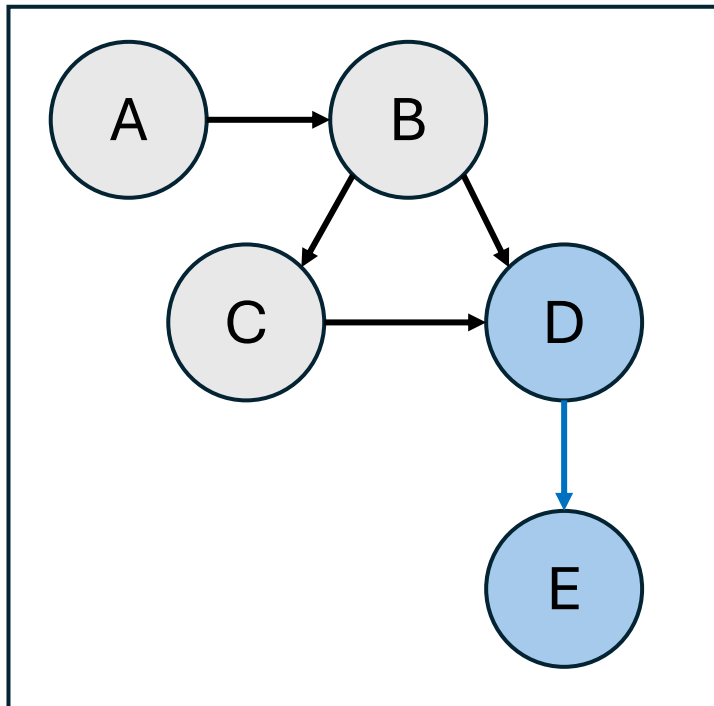
A controlled, three-step agent evaluation framework

- We introduce **CABRA**¹ to generate code editing tasks from scratch 🐐

Step 1: Turn directed acyclic graphs into programs

Step 2: Transform them into five tasks [[Fowler's Refactors](#)] → e.g., add input parameter

Directed Acyclic Graph



Generated Program

```
def A(val: float) -> float:
    return math.sin(B(val))

def B(val: float) -> float:
    return C(val) * D(val)

def C(val: float) -> float:
    return -1.0 * D(val)

def D(val: float, m: float) -> float:
    return 0.5 + E(val)

def E(val: float, m: float) -> float:
    return m * math.sqrt(val)
```

A controlled, three-step agent evaluation framework

Step 3: We fix the codebases + tasks (“Add a Parameter”) and test diverse agent abilities

- We design four axes that are controllable by a task size integer **N**

Function Traversal: Track **N** functions

```
1 def add_param_1(x)
2   return add_param_2(x)
3 def add_param_2(x)
4   return add_param_3(x)
5 ...
6 def add_param_N(x) ...
```

Function Search: Navigate **N** distractors

```
1 def distractor_func_1(x)
2 def distractor_func_2(x)
3 ...
4 def add_param_start(x)
5 ...
6 def distractor_func_N(x)
```

Runtime Analysis: Trace a val for **N** steps

Only make changes if this is true:

```
1 def check(x: int, N: int):
2   for step in range(N):
3     x = ...
4   return x % 2 == 0
```

Instruction Following: Adhere to **N** rules

Follow these **N** rules while editing:

1. Assert all values are less than 5
2. Add the comment “# hello”
- ...
- N. Store `mult` in a temp variable

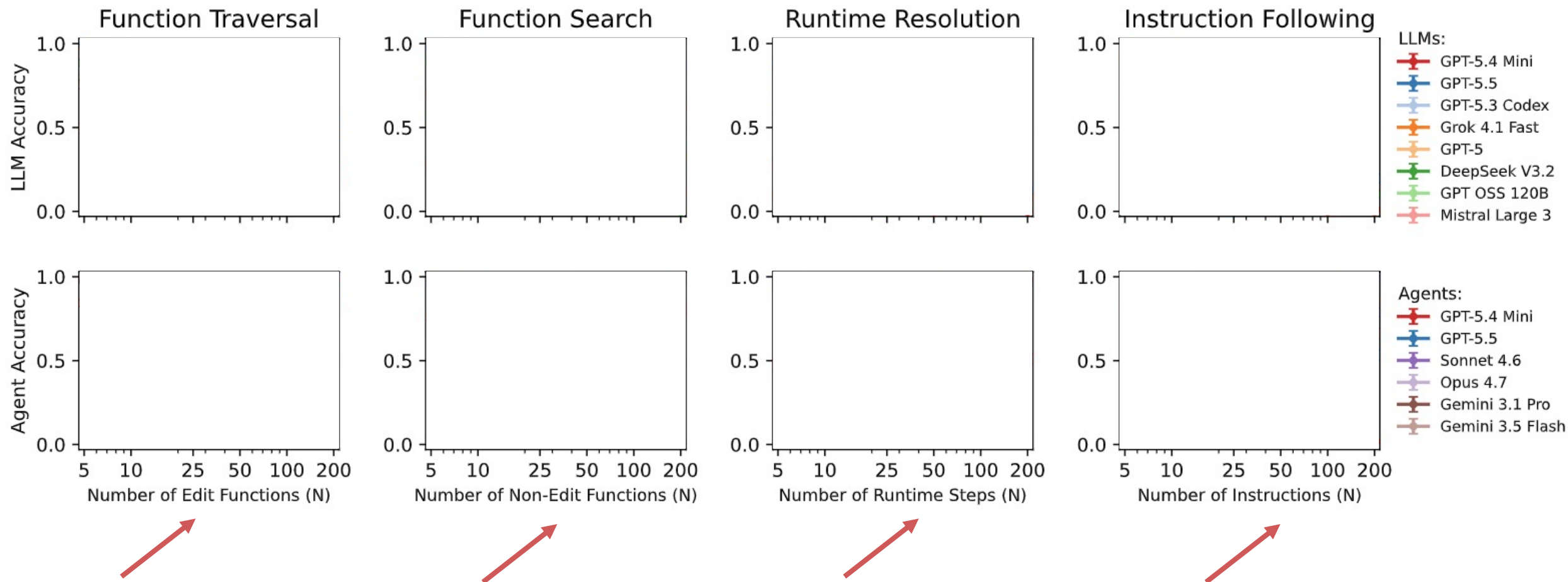


We evaluate eight LLMs without tools + six coding agents (Copilot harness)

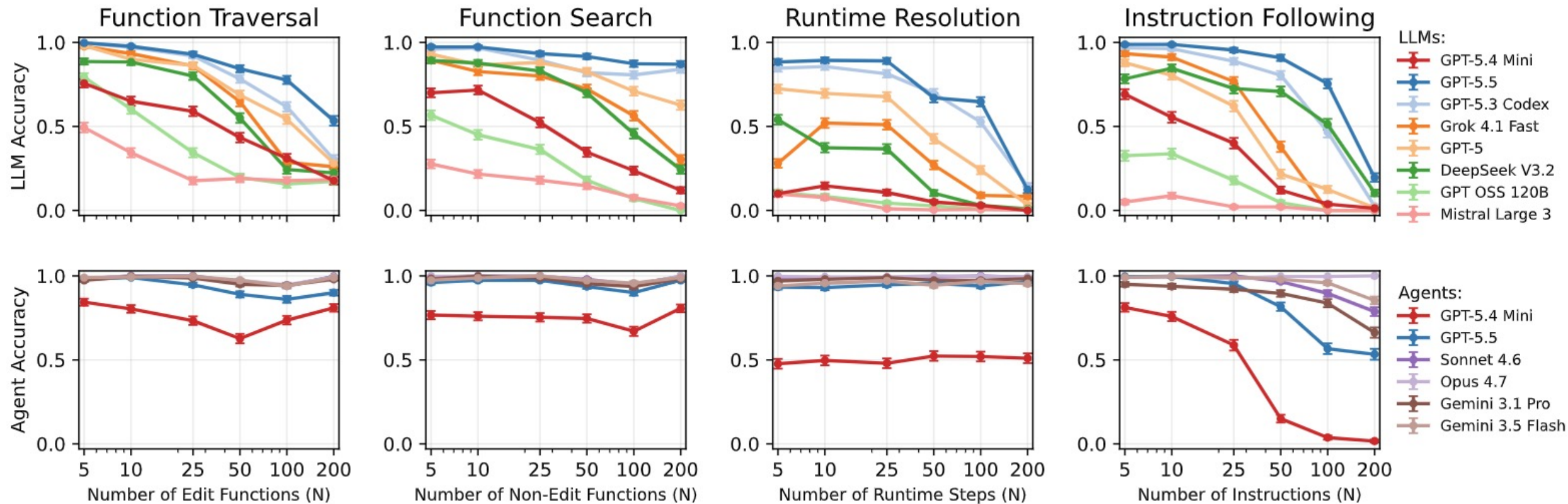


We evaluate eight LLMs without tools + six coding agents (Copilot harness)

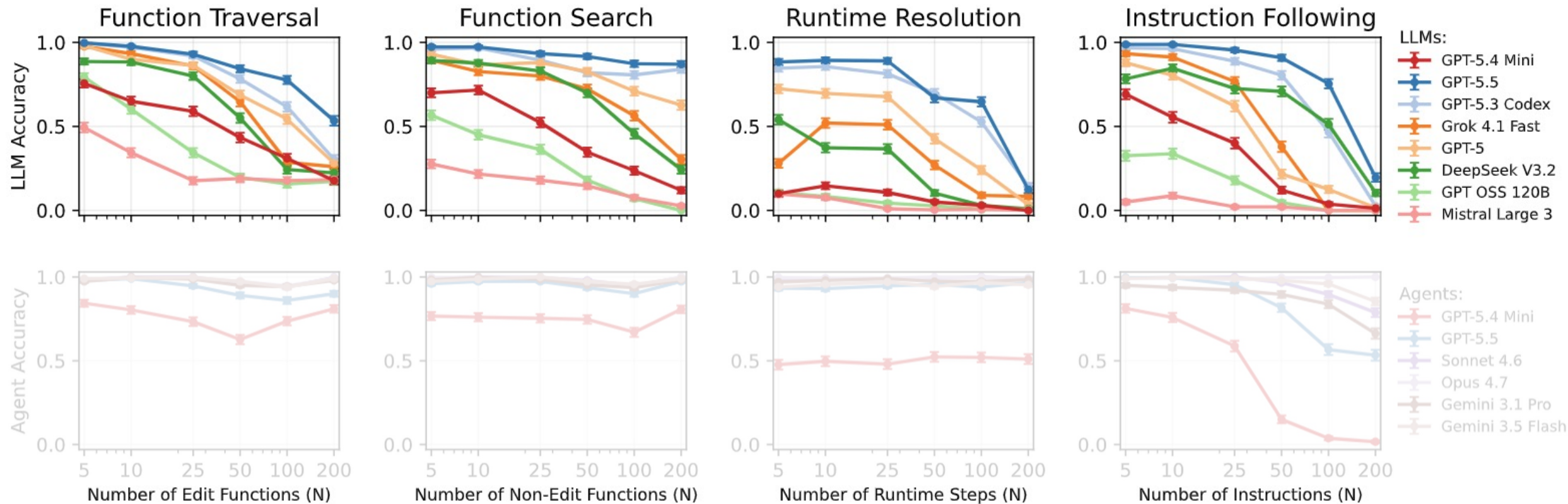
- We generate 6,840 total CABRA tasks with sizes $n \in \{5, 10, 25, 50, 100, 200\}$



We evaluate eight LLMs without tools + six coding agents (Copilot harness)

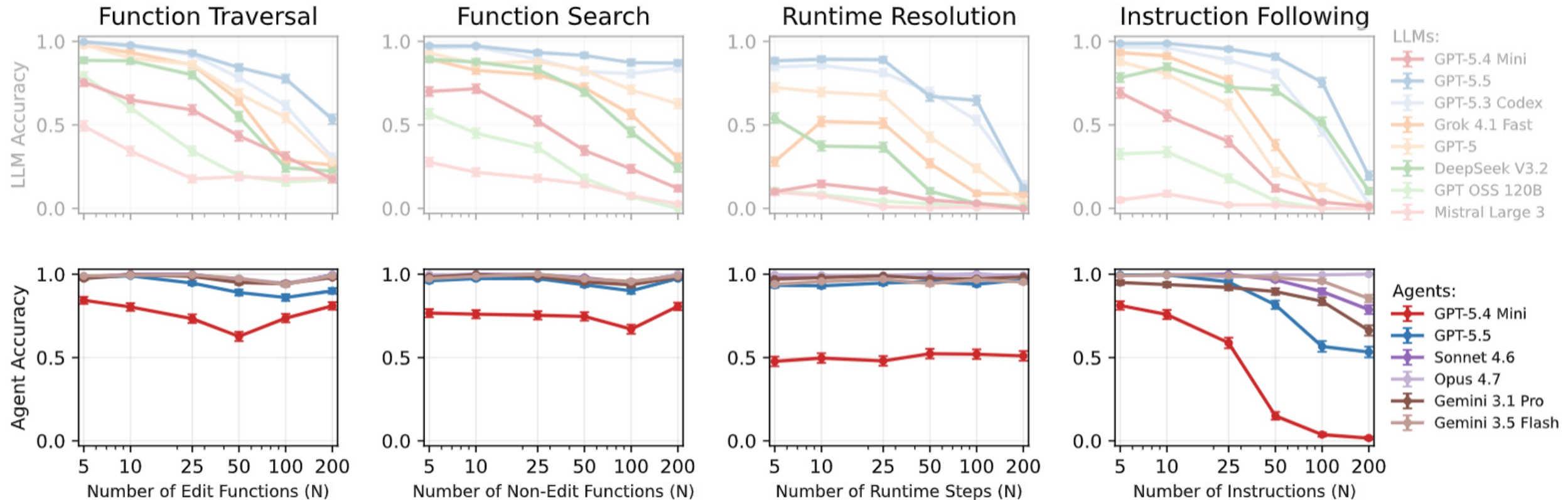


We evaluate eight LLMs without tools + six coding agents (Copilot harness)



➤ Larger tasks lead to lower accuracy for LLMs!

We evaluate eight LLMs without tools + six coding agents (Copilot harness)



➤ Larger tasks lead to lower accuracy for LLMs!

➤ But agents stay constant, and often near perfect... Why???



Why??? 🥲 Agents ace our tasks thanks to tools



Hey Copilot, please make these 250 code edits

Lol no, I'll write a script that does the 250 edits

```
1 def apply_edits():  
2 # same code I used to make the task
```



Work for coding agents isn't just # edits...

Since they can just offload work to *tools* that make edits

Since tools are so important, which *types* of tools do agents use?

- We review agent traces and discover five high-level reasons they invoke tools

Since tools are so important, which *types* of tools do agents use?

- We review agent traces and discover five high-level reasons they invoke tools

Read: View exact lines of code

“I'm starting by looking at the structure of the solution.py file to understand what I'm working with.”

```
bash
```

```
find /home/t-nbalepur/agent_scratch/claude-opus-  
4_7/extract_helper/tmpvrb_vzx4 -type f | head -20
```

Since tools are so important, which *types* of tools do agents use?

- We review agent traces and discover five high-level reasons they invoke tools

Read: View exact lines of code

Analyze: Study call graphs, etc.

“I’m tracing which functions transitively call `external` by working through the call graph in rounds...”

bash

```
cd /home/t-nbalepur/agent_scratch/claude-opus-4_7/add_return_value/tmpkrj_8ldh/ && python3 -c "
import ast
with open('solution.py') as f:
    tree = ast.parse(f.read())
funcs = {}
for node in ast.walk(tree):
    if isinstance(node, ast.FunctionDef):
        calls = set()
        for sub in ast.walk(node):
            if isinstance(sub, ast.Call) and isinstance(sub.func,
```


Since tools are so important, which *types* of tools do agents use?

- We review agent traces and discover five high-level reasons they invoke tools

Read: View exact lines of code

Analyze: Study call graphs, etc.

Search: Find exact words / lines

“I’m verifying that the refactoring is correct: `slow()` appears only once inside `ax()`...”

bash

```
cd /home/t-nbalepur/agent_scratch/claude-opus-4_7/cache_function/tmp66ziom7d && grep -n "slow()" solution.py
```


Since tools are so important, which *types* of tools do agents use?

- We review agent traces and discover five high-level reasons they invoke tools

Read: View exact lines of code

Analyze: Study call graphs, etc.

Search: Find exact words / lines

Edit: Make changes to the code

“Only `ab` has the `mult` function, so I need to trace through its call graph and propagate changes downstream....”

edit

```
old str: def ab(val: float, mult: float=default()) -> float:
    at_val = at(val)
    bc_val = bc(val)
    bl_val = bl(val)
    return operator.add(math.copysign(at_val, bc_val), bl_val)
new str: def ab(val: float, mult: float=default()) -> float:
    at_val = at(val, mult=mult)
    bc_val = bc(val, mult=mult)
    bl_val = bl(val, mult=mult)
    return operator.add(math.copysign(at_val, bc_val), bl_val)
```

Since tools are so important, which *types* of tools do agents use?

- We review agent traces and discover five high-level reasons they invoke tools

Read: View exact lines of code

Analyze: Study call graphs, etc.

Search: Find exact words / lines

Edit: Make changes to the code

Test: Verify the generated edits

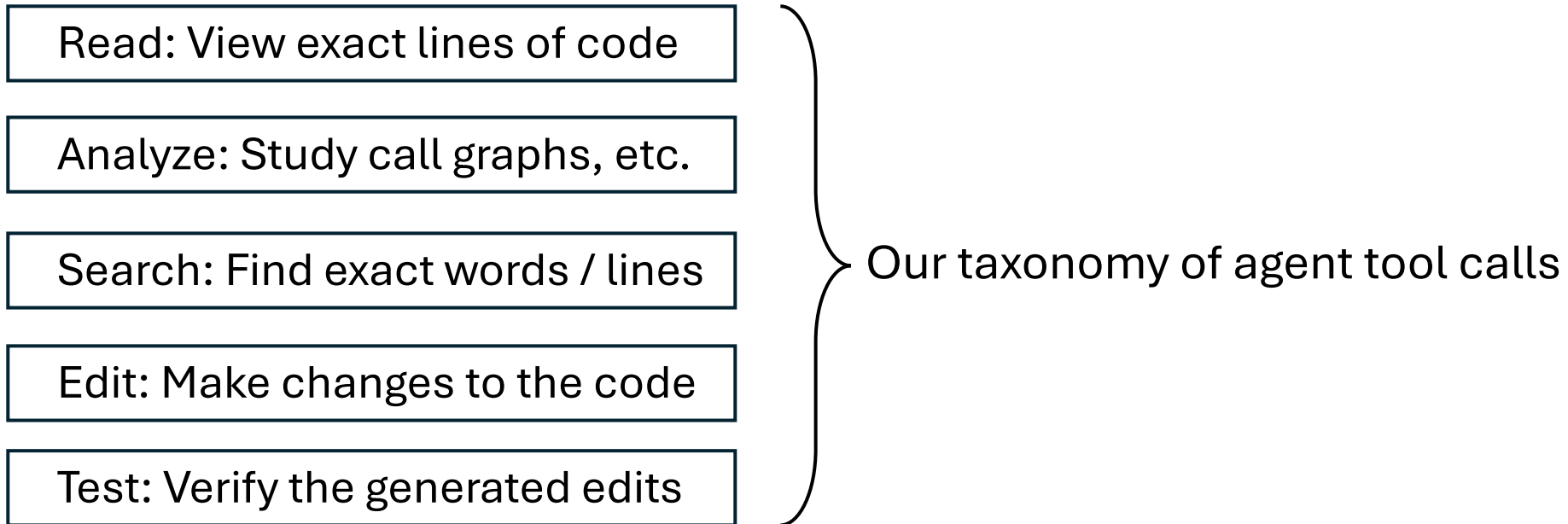
“I’ll create a simple test to validate the end-to-end behavior...”

bash

```
cd /tmp && mkdir -p test_solution && cd test_solution && cp /home/t-  
nbalepur/agent_scratch/claude-opus-  
4_7/add_return_value/tmpi5vog4li/solution.py . && cat > mylibrary.py  
<<'EOF'  
def external(val: str) -> tuple[str, str]:  
    return (val + "_ext", "PROPAGATED_KEY")  
EOF  
cat > strops.py <<'EOF'  
def rot13(val): return val + "_rot13"  
def char_rotate_by_3(val): return val + "_r3"  
def char_rotate(val): return val + "_cr"
```

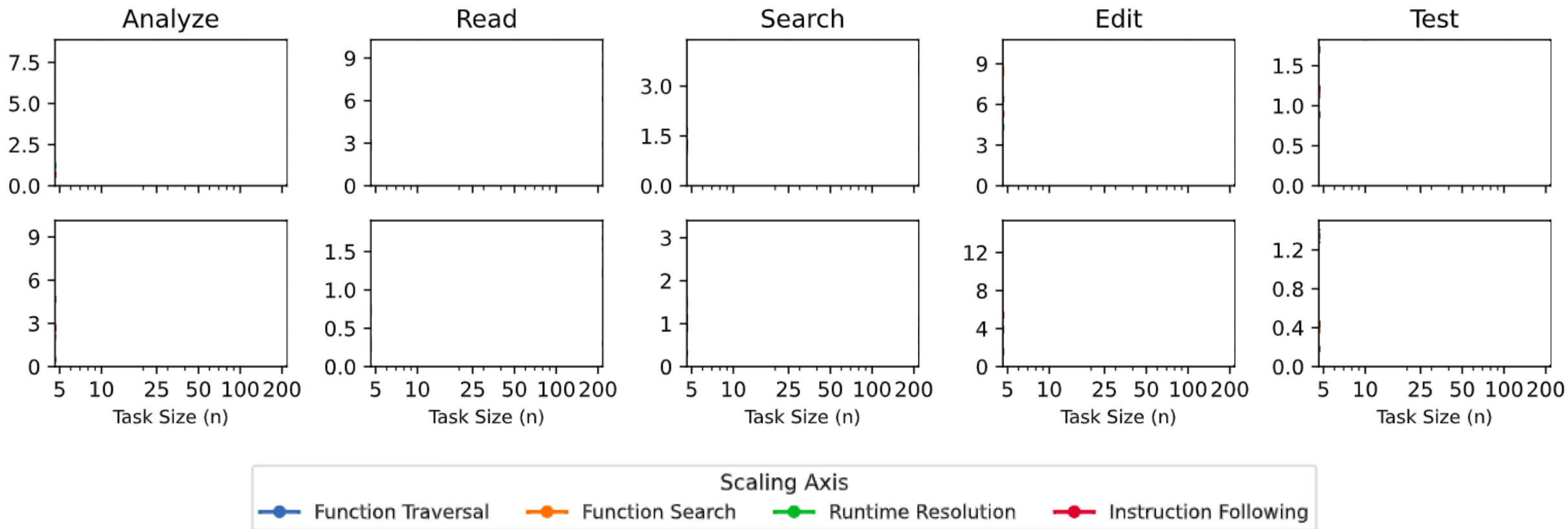
Since tools are so important, which *types* of tools do agents use?

- We review agent traces and discover five high-level reasons they invoke tools



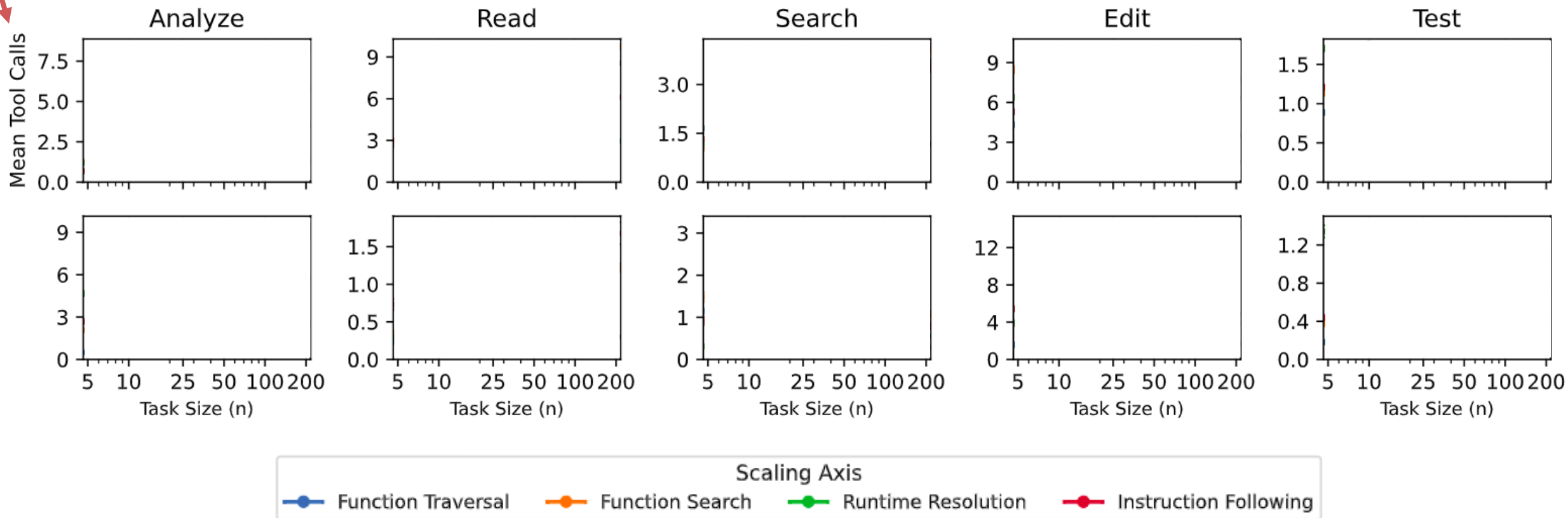
- We then prompt Qwen-4B to classify each tool type agents use (~900k tool calls...)

Where do agents spend effort?



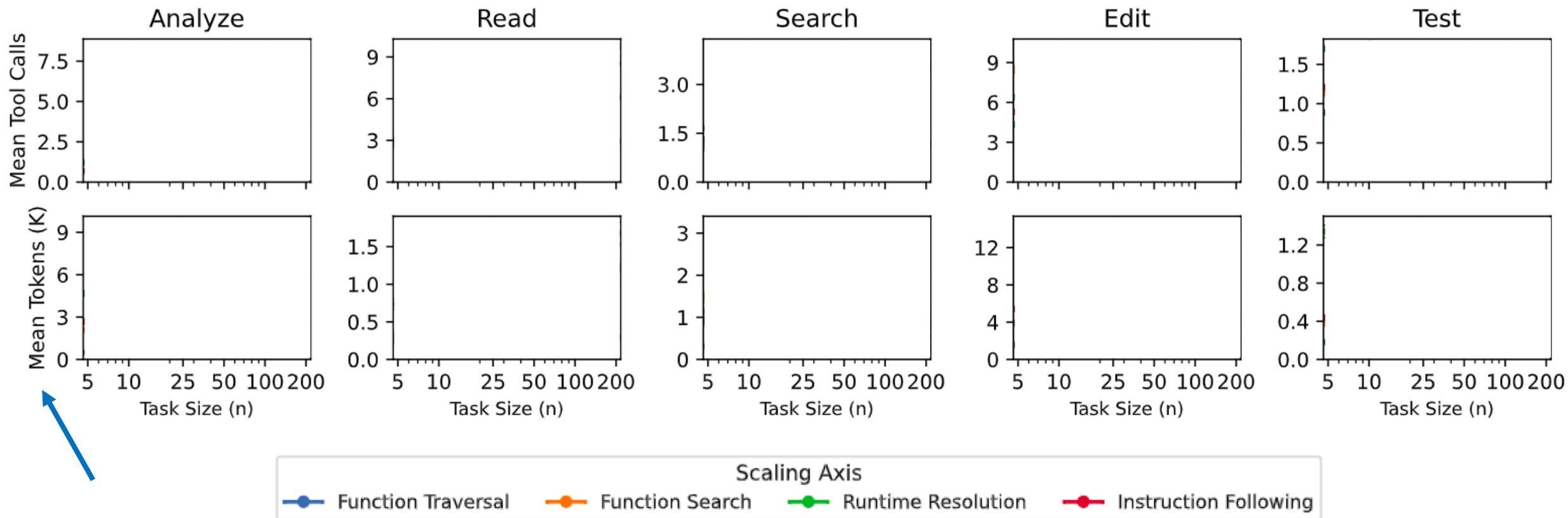
Where do agents spend effort?

➤ We report: 1) the **total #** of each tool call

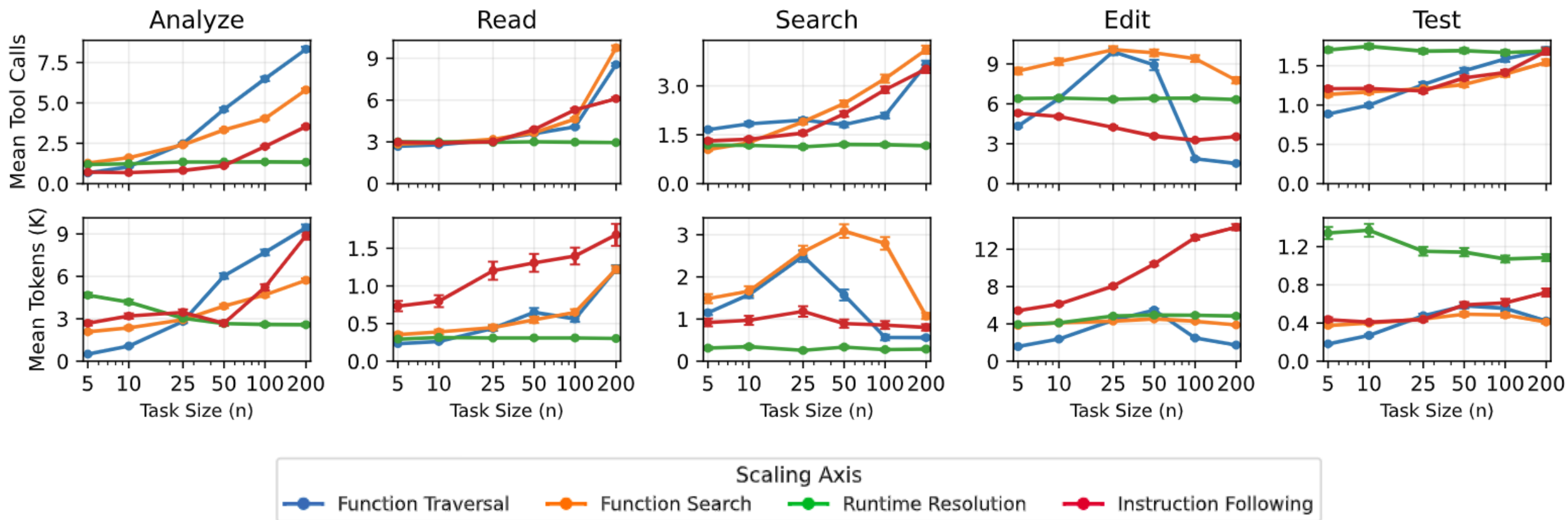


Where do agents spend effort?

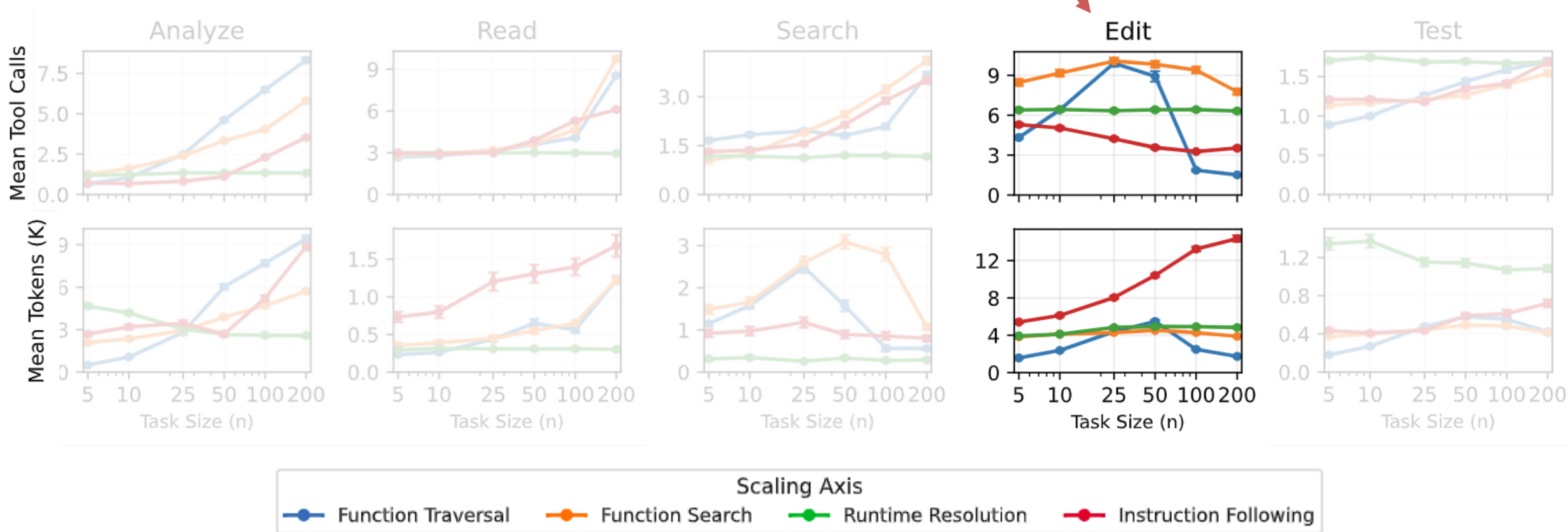
- We report: 1) the total # of each tool call; and 2) # tokens used to generate each tool call



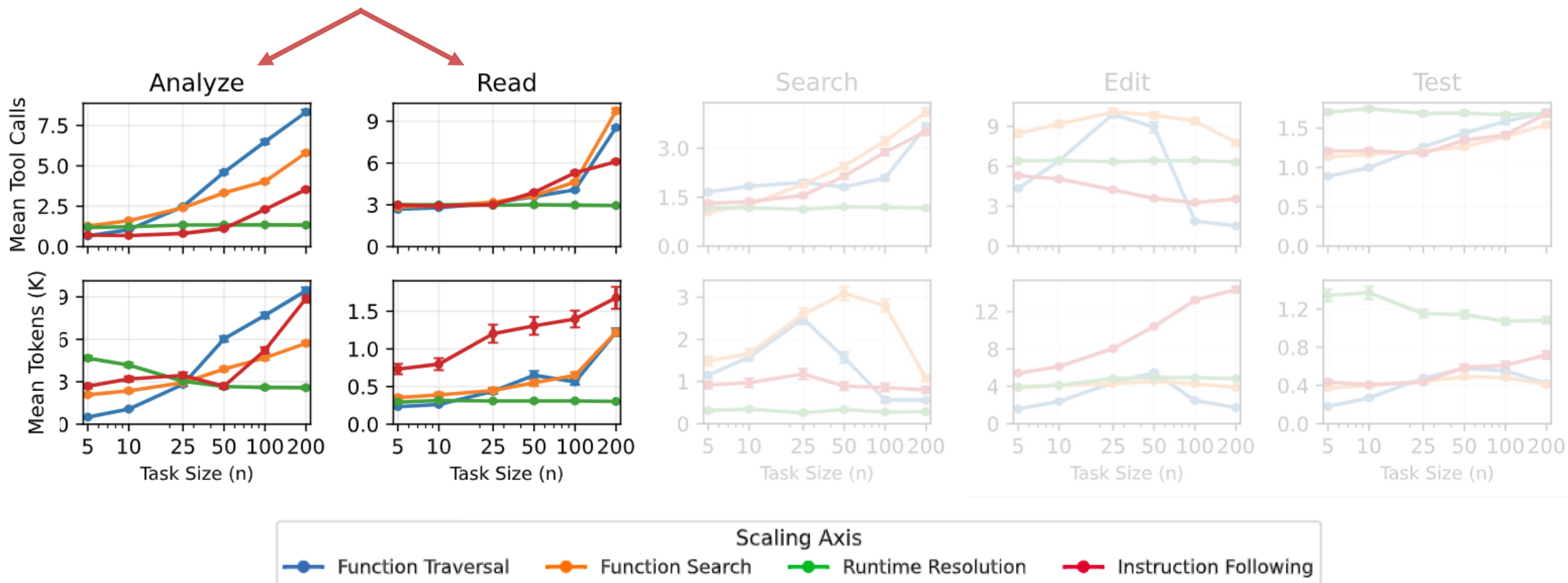
Where do agents spend effort?



Where do agents spend effort? Not *editing*.



Where do agents spend effort? Not *editing*. But **understanding!**



➤ **Analyze + Read** tool calls almost always happen right before edits

Extending **CABRA** to target understanding: merge codebases task

Extending **CABRA** to target understanding: merge codebases task

- Sample two programs with identical logic

```
class CalculateA(Calculate):  
    def custom(v1, v2):
```

```
class CalculateB(Calculate):  
    def custom(v1, v2):
```

Extending **CABRA** to target understanding: merge codebases task

- Sample two programs with identical logic

```
class CalculateA(Calculate):  
    def custom(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = 2 * v3 + 3 * v4  
        v6 = v5 - v4  
        return v6 + v3
```

```
class CalculateB(Calculate):  
    def custom(v1, v2):
```

Extending **CABRA** to target understanding: merge codebases task

- Sample two programs with identical logic

```
class CalculateA(Calculate):  
    def custom(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = 2 * v3 + 3 * v4  
        v6 = v5 - v4  
        return v6 + v3
```

```
class CalculateB(Calculate):  
    def custom(v1, v2):  
        v4 = v3 + v2  
        v3 = v1 + v1 - v2  
        v5 = 2 * v3 + 3 * v4  
        return v5 - v4 + v3
```

Extending **CABRA** to target understanding: merge codebases task

- Sample two programs with identical logic

```
class CalculateA(Calculate):  
    def custom(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = 2 * v3 + 3 * v4  
        v6 = v5 - v4  
        return v6 + v3
```

```
class CalculateB(Calculate):  
    def custom(v1, v2):  
        v4 = v3 + v2  
        v3 = v1 + v1 - v2  
        v5 = 2 * v3 + 3 * v4  
        return v5 - v4 + v3
```

Create equivalent lines by:

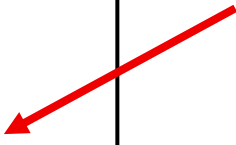
- Renaming variables
- Decomposing lines
- Shuffling lines
- ...

Extending **CABRA** to target understanding: merge codebases task

- Sample two programs with identical logic, then inject **one difference**

```
class CalculateA(Calculate):  
    def custom(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = 2 * v3 + 3 * v4  
        v6 = v5 - v4  
        return v6 + v3
```

```
class CalculateB(Calculate):  
    def custom(v1, v2):  
        v4 = v3 + v2  
        v3 = v1 + v1 - v2  
        v5 = 3 * v3 + 3 * v4  
        return v5 - v4 + v3
```



Extending **CABRA** to target understanding: merge codebases task

- Sample two programs with identical logic, then inject one difference

```
class CalculateA(Calculate):  
    def custom(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = 2 * v3 + 3 * v4  
        v6 = v5 - v4  
        return v6 + v3
```

```
class CalculateB(Calculate):  
    def custom(v1, v2):  
        v4 = v3 + v2  
        v3 = v1 + v1 - v2  
        v5 = 3 * v3 + 3 * v4  
        return v5 - v4 + v3
```

```
class Calculate:  
    def shared(v1, v2):
```

```
class CalculateA(Calculate):  
    def custom(v3, v4):
```

```
class CalculateB(Calculate):  
    def custom(v3, v4):
```


Extending **CABRA** to target understanding: merge codebases task

- Sample two programs with identical logic, then inject one difference
- Instruct models to extract the shared logic into a parent class

```
class CalculateA(Calculate):  
    def custom(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = 2 * v3 + 3 * v4  
        v6 = v5 - v4  
        return v6 + v3
```

```
class CalculateB(Calculate):  
    def custom(v1, v2):  
        v4 = v3 + v2  
        v3 = v1 + v1 - v2  
        v5 = 3 * v3 + 3 * v4  
        return v5 - v4 + v3
```

```
class Calculate:  
    def shared(v1, v2):
```

```
class CalculateA(Calculate):  
    def custom(v3, v4):
```

```
class CalculateB(Calculate):  
    def custom(v3, v4):
```

Extending **CABRA** to target understanding: merge codebases task

- Sample two programs with identical logic, then inject one difference
- Instruct models to extract the shared logic into a parent class

```
class CalculateA(Calculate):  
    def custom(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = 2 * v3 + 3 * v4  
        v6 = v5 - v4  
        return v6 + v3
```

```
class CalculateB(Calculate):  
    def custom(v1, v2):  
        v4 = v3 + v2  
        v3 = v1 + v1 - v2  
        v5 = 3 * v3 + 3 * v4  
        return v5 - v4 + v3
```

```
class Calculate:  
    def shared(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = custom(v3, v4)  
        return v5 - v4 + v3
```

```
class CalculateA(Calculate):  
    def custom(v3, v4):
```

```
class CalculateB(Calculate):  
    def custom(v3, v4):
```

Extending **CABRA** to target understanding: merge codebases task

- Sample two programs with identical logic, then inject one difference
- Instruct models to extract the shared logic into a parent class, while preserving the difference

```
class CalculateA(Calculate):  
    def custom(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = 2 * v3 + 3 * v4  
        v6 = v5 - v4  
        return v6 + v3
```

```
class CalculateB(Calculate):  
    def custom(v1, v2):  
        v4 = v3 + v2  
        v3 = v1 + v1 - v2  
        v5 = 3 * v3 + 3 * v4  
        return v5 - v4 + v3
```

```
class Calculate:  
    def shared(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = custom(v3, v4)  
        return v5 - v4 + v3
```

```
class CalculateA(Calculate):  
    def custom(v3, v4):
```

```
class CalculateB(Calculate):  
    def custom(v3, v4):
```

Extending **CABRA** to target understanding: merge codebases task

- Sample two programs with identical logic, then inject one difference
- Instruct models to extract the shared logic into a parent class, while preserving the difference

```
class CalculateA(Calculate):  
    def custom(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = 2 * v3 + 3 * v4  
        v6 = v5 - v4  
        return v6 + v3
```

```
class CalculateB(Calculate):  
    def custom(v1, v2):  
        v4 = v3 + v2  
        v3 = v1 + v1 - v2  
        v5 = 3 * v3 + 3 * v4  
        return v5 - v4 + v3
```

```
class Calculate:  
    def shared(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = custom(v3, v4)  
        return v5 - v4 + v3
```

```
class CalculateA(Calculate):  
    def custom(v3, v4):  
        return 2 * v3 + 3 * v4
```

```
class CalculateB(Calculate):  
    def custom(v3, v4):  
        return 3 * v3 + 3 * v4
```

Extending **CABRA** to target understanding: merge codebases task

- Sample two programs with identical logic, then inject one difference
- Instruct models to extract the shared logic into a parent class, while preserving the difference

```
class CalculateA(Calculate):  
    def custom(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = 2 * v3 + 3 * v4  
        v6 = v5 - v4  
        return v6 + v3
```

```
class CalculateB(Calculate):  
    def custom(v1, v2):  
        v4 = v3 + v2  
        v3 = v1 + v1 - v2  
        v5 = 3 * v3 + 3 * v4  
        return v5 - v4 + v3
```

```
class Calculate:  
    def shared(v1, v2):  
        v3 = v1 + v1 - v2  
        v4 = v3 + v2  
        v5 = custom(v3, v4)  
        return v5 - v4 + v3
```

```
class CalculateA(Calculate):  
    def custom(v3, v4):  
        return 2 * v3 + 3 * v4
```

```
class CalculateB(Calculate):  
    def custom(v3, v4):  
        return 3 * v3 + 3 * v4
```

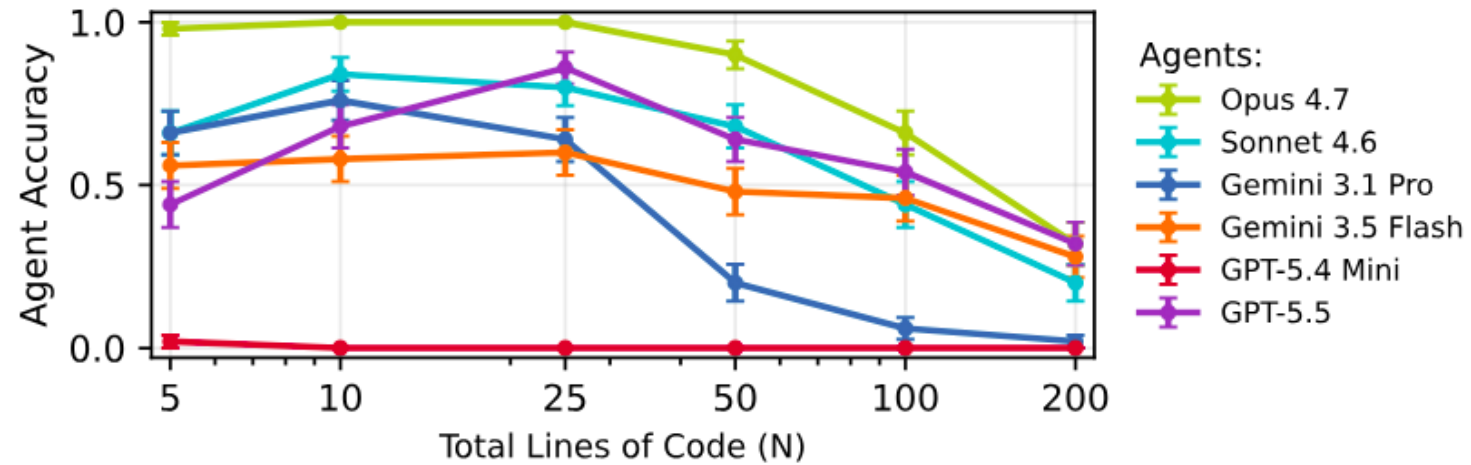
Intuitively, models may not edit that much code (e.g., copy and paste lines)

But they must compare and understand which of many lines are identical

This task supports our hypothesis: code understanding is a weakness

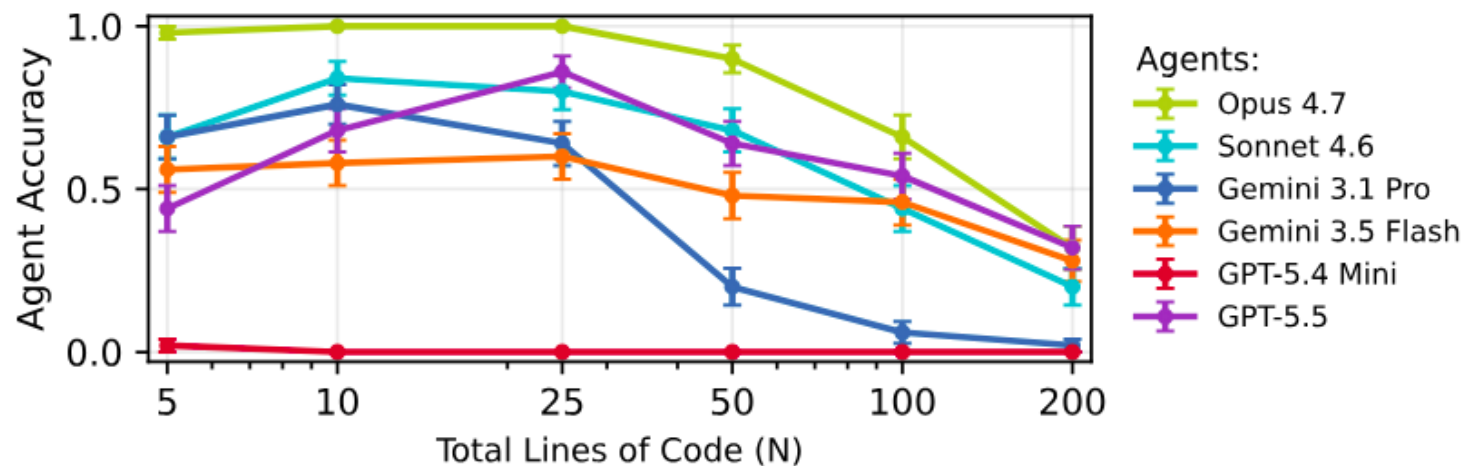
This task supports our hypothesis: code understanding is a weakness

- Agent accuracy finally crumbles with **N!** 100

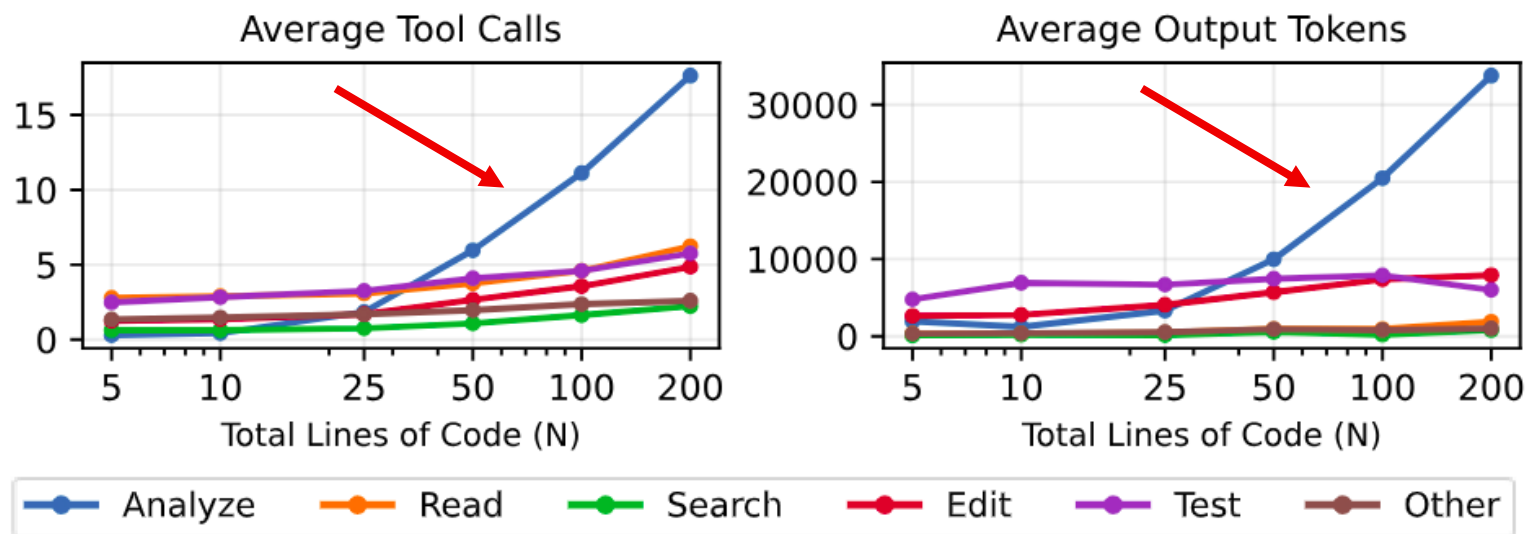


This task supports our hypothesis: code understanding is a weakness

- Agent accuracy finally crumbles with **N!** ¹⁰⁰



- And the task does elicit far more **Analyze** tool calls relative to **Edit** 🚀



Takeaways!



Things that are hard for LLMs can become trivial with tool use



Tool use as a lens for formalizing + studying agent behavior



Operationalizing understanding is a clear step for coding agents



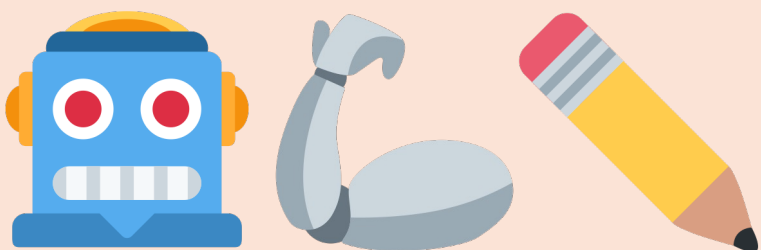
Pause for Questions



To make “progress”, we get agents to edit more and more code...

But this motivates two questions for **offline** + online evals:

RQ1: Which abilities beyond editing do coding agents need, but lack?



Programming is Not Just Editing! The Importance of Code Understanding for Agents (Under Review)

RQ2: Does optimizing agents for editing support human developers?



(Im)Paired Programming: Coding Agents Improve Productivity but Harm Understanding (Under Review)



The common link? Understanding is a weakness! 🧠

To make “progress”, we get agents to edit more and more code...

But this motivates two questions for **offline** + **online** evals:

RQ1: Which abilities beyond editing do coding agents need, but lack?



Programming is Not Just Editing! The Importance of Code Understanding for Agents (Under Review)

RQ2: Does optimizing agents for editing support human developers?



(Im)Paired Programming: Coding Agents Improve Productivity but Harm Understanding (Under Review)



The common link? Understanding is a weakness! 🧠

Motivation: Understanding is very important!

- NLP benchmarks focus on task completion, but programmers don't just complete tasks
- They also need to *understand* their code: know its content + behavior
 - It's estimated that 70% of programming work is understanding [[Minelli, 2015](#)]
 - 19% of tasks in public human-agent interactions relate to understanding [[SWE-Chat](#)]
- When people write code they understand it, but with agents, we don't write code! 🤖

We thus ask:

Do users with agents understand their code?

A 3-step study design: Do users with agents understand their code?

Study Design on Code Understanding

Background Quiz (\$2.1)

Warm-Up

Initial Website Task (\$2.2)

Warm-Up

Comprehension Test (\$2.3)

Extension Task (\$2.4)

Text Feedback

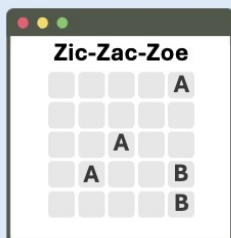
Group A: **Agent** directly edits the user's code

Add a header on top



I changed the `index.html` file to use `<h1>` tags:

```
1 <div id="content">
+ <h1>Zic-Zac-Zoe</h1>
2 <div id="board">
3 ...
4 </div>
5 </div>
```



Group B: **Chatbot** w/ high-level syntax – users write code

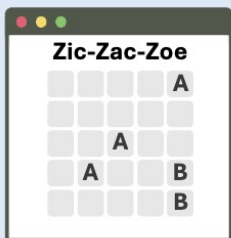
Add a header on top



You can add this using `<h1>` tags:

```
<h1>title</h1>
```

```
1 <div id="content">
+ <h1>Zic-Zac-Zoe</h1>
2 <div id="board">
3 ...
4 </div>
5 </div>
```



RQ1: Do users **comprehend** the code they create with AI agents?

What would happen if `id` changed?

```
1 <div id="content">...</div>
```

(A) The board would disappear

...

(A) The board would no longer update



Which code snippets is yours?

```
* <div id="board">
2 ...
3 </div>
```

Left is Mine



```
* <div id="grid">
2 ...
3 </div>
```

Right is Mine

RQ2: Can users **extend** their initial code without AI agents?

How can I add a reset button?

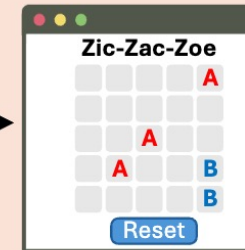


Here's the syntax for a `<button>`:

```
<button>name</button>
```



```
1 <div class="content">
2 <h1>Zic-Zac-Zoe</h1>
3 <div id="board">
4 ...
5 </div>
6 </div>
+ <button id="reset">
+ Reset</but I
```



Step 1: Users make a website to pass tests w/ an agent or a chatbot

Study Design on Code Understanding

Background Quiz (\$2.1)

Warm-Up

Initial Website Task (\$2.2)

Warm-Up

Comprehension Test (\$2.3)

Extension Task (\$2.4)

Text Feedback

Group A: **Agent** directly edits the user's code

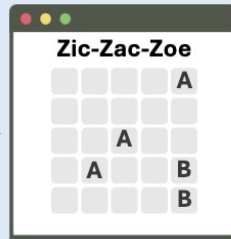
Add a header on top



I changed the `index.html` file to use `<h1>` tags:



```
1 <div id="content">
+ <h1>Zic-Zac-Zoe</h1>
2 <div id="board">
3 ...
4 </div>
5 </div>
```



Group B: **Chatbot** w/ high-level syntax – users write code

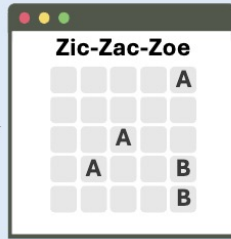
Add a header on top



You can add this using `<h1>` tags:



```
1 <div id="content">
+ <h1>Zic-Zac-Zoe</h1>
2 <div id="board">
3 ...
4 </div>
5 </div>
```



RQ1: Do users **comprehend** the code they create with AI agents?

What would happen if `id` changed?

```
1 <div id="content">...</div>
```

(A) The board would disappear

...

(A) The board would no longer update



Which code snippets is yours?

```
* <div id="board">
2 ...
3 </div>
```

Left is Mine



```
* <div id="grid">
2 ...
3 </div>
```

Right is Mine

RQ2: Can users **extend** their initial code without AI agents?

How can I add a reset button?



Here's the syntax for a `<button>`:



```
<button>name</button>
```



```
1 <div class="content">
2 <h1>Zic-Zac-Zoe</h1>
3 <div id="board">
4 ...
5 </div>
6 </div>
+ <button id="reset">
+ Reset</but I
```

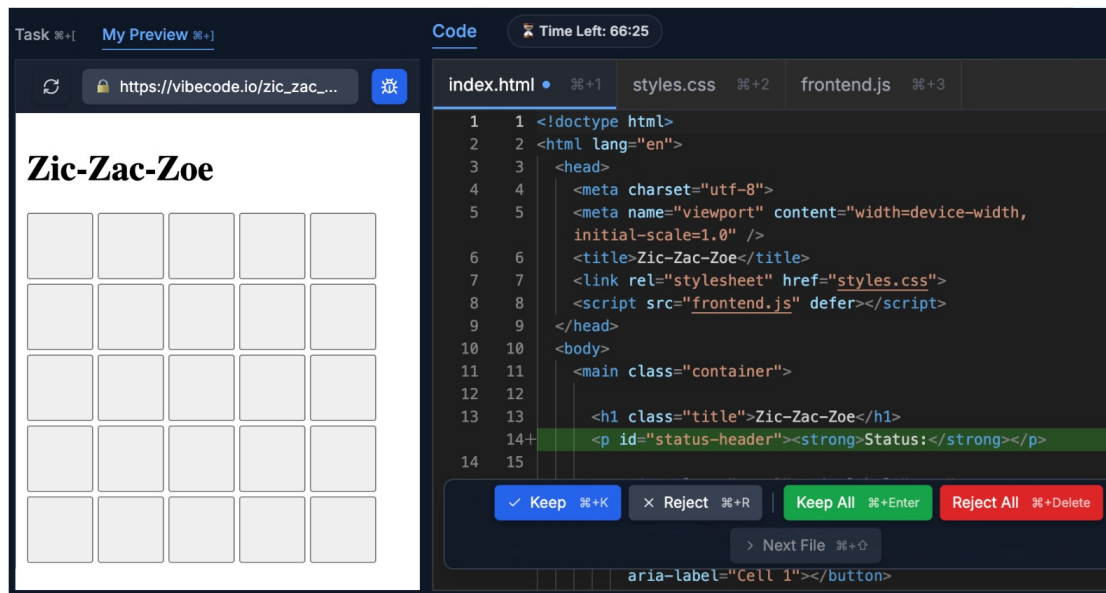


Step 1: Users make a website to pass tests w/ an agent or a chatbot

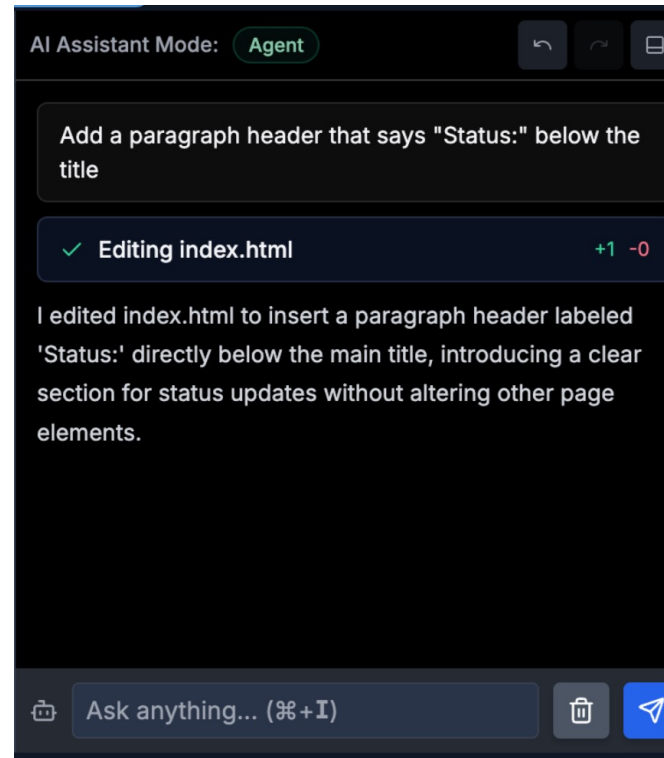
We tell users to implement a fictional game called “zic-zac-zoe” (familiar but novel):

1. 5x5 board, centered horizontally
2. Players make 2 moves at a time (A -> A -> B -> B)
3. Players win if they get a row or column

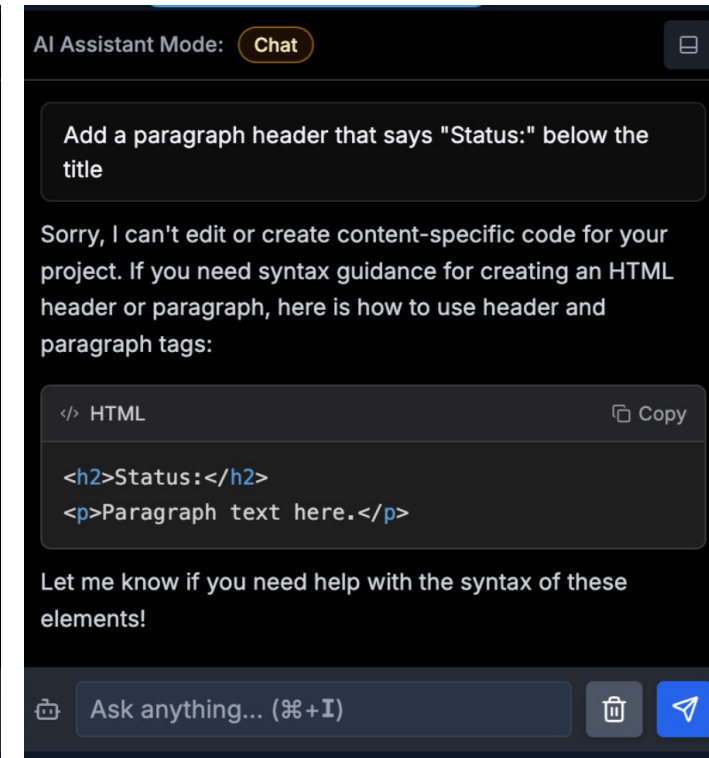
All users can view the code + website



Agents: Prompt + Review



Chat: Users write code



Step 1: Users make a website to pass tests w/ an agent or a chatbot

Study Design on Code Understanding

Background Quiz (\$2.1)

Warm-Up

Initial Website Task (\$2.2)

Warm-Up

Comprehension Test (\$2.3)

Extension Task (\$2.4)

Text Feedback

Group A: **Agent** directly edits the user's code

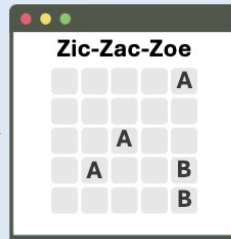
Add a header on top



I changed the `index.html` file to use `<h1>` tags:



```
1 <div id="content">
+ <h1>Zic-Zac-Zoe</h1>
2 <div id="board">
3 ...
4 </div>
5 </div>
```



Group B: **Chatbot** w/ high-level syntax – users write code

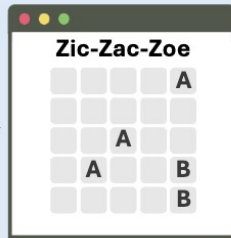
Add a header on top



You can add this using `<h1>` tags:



```
1 <div id="content">
+ <h1>Zic-Zac-Zoe</h1>
2 <div id="board">
3 ...
4 </div>
5 </div>
```



RQ1: Do users **comprehend** the code they create with AI agents?

What would happen if `id` changed?

```
1 <div id="content">...</div>
```

(A) The board would disappear

...

(A) The board would no longer update



Which code snippets is yours?

```
* <div id="board">
2 ...
3 </div>
```

Left is Mine



```
* <div id="grid">
2 ...
3 </div>
```

Right is Mine

RQ2: Can users **extend** their initial code without AI agents?

How can I add a reset button?



Here's the syntax for a `<button>`:

```
<button>name</button>
```



```
1 <div class="content">
2 <h1>Zic-Zac-Zoe</h1>
3 <div id="board">
4 ...
5 </div>
6 </div>
+ <button id="reset">
+ Reset</but I
```



Step 2: Answer comprehension questions (understanding metric 1)

Study Design on Code Understanding

Background Quiz (\$2.1)

Warm-Up

Initial Website Task (\$2.2)

Warm-Up

Comprehension Test (\$2.3)

Extension Task (\$2.4)

Text Feedback

Group A: **Agent** directly edits the user's code

Add a header on top



I changed the
index.html file
to use `<h1>` tags:

```
1 <div id="content">
+ <h1>Zic-Zac-Zoe</h1>
2 <div id="board">
3 ...
4 </div>
5 </div>
```



Group B: **Chatbot** w/ high-level syntax – users write code

Add a header on top



You can add this
using `<h1>` tags:

```
<h1>title</h1>
```

```
1 <div id="content">
+ <h1>Zic-Zac-Zoe</h1>
2 <div id="board">
3 ...
4 </div>
5 </div>
```



RQ1: Do users **comprehend** the code they create with AI agents?

What would happen if `id` changed?

```
1 <div id="content">...</div>
```

(A) The board would disappear

...

(A) The board would no longer update



Which code snippets is yours?

```
* <div id="board">
2 ...
3 </div>
```

Left is Mine



```
* <div id="grid">
2 ...
3 </div>
```

Right is Mine

RQ2: Can users **extend** their initial code without AI agents?

How can I add a reset button?

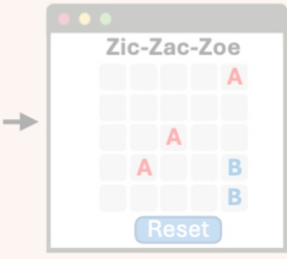


Here's the syntax for a `<button>`:

```
<button>name</button>
```



```
1 <div class="content">
2 <h1>Zic-Zac-Zoe</h1>
3 <div id="board">
4 ...
5 </div>
6 </div>
+ <button id="reset">
+ Reset</but I
```



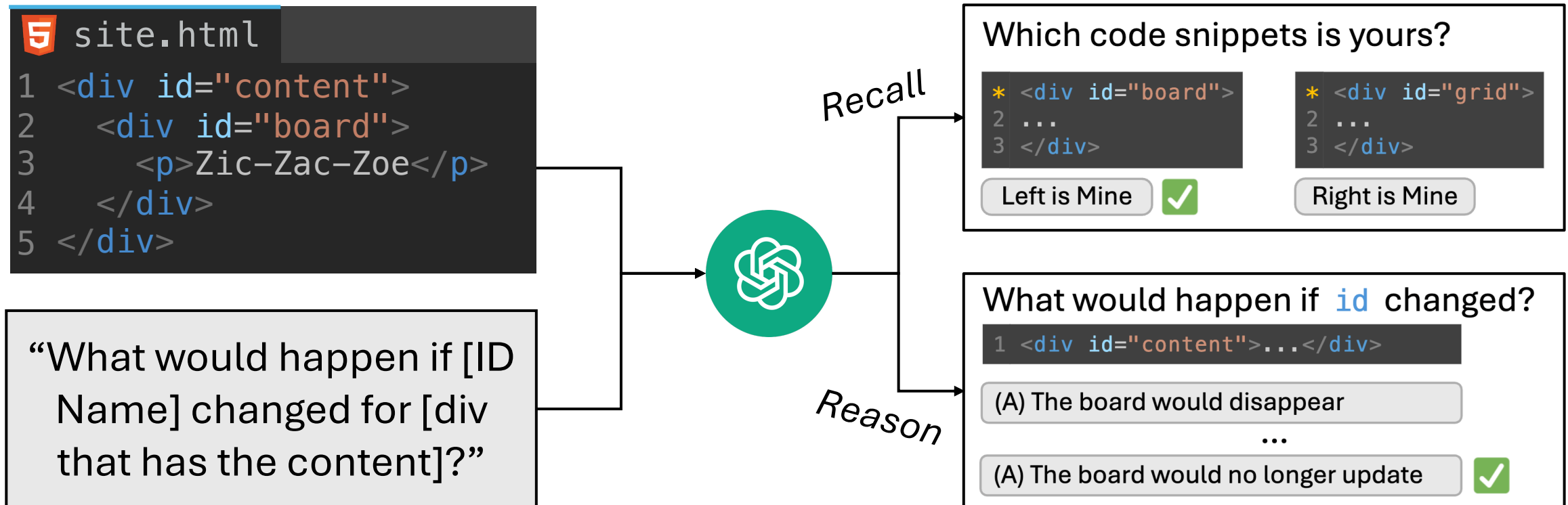
Step 2: Answer comprehension questions (understanding metric 1)

We show users code snippets + ask two question types, synthesized from [Wyrich et al., 2023](#)

1. Recall: Users select which items exist exactly in their code
2. Reasoning: Users answer the purpose and behavior of their code

One thing that is hard: everyone submits different code!

➤ To personalize, we define question templates + prompt LLMs to tailor it to users' code



Step 2: Answer comprehension questions (understanding metric 1)

Study Design on Code Understanding

Background Quiz (\$2.1)

Warm-Up

Initial Website Task (\$2.2)

Warm-Up

Comprehension Test (\$2.3)

Extension Task (\$2.4)

Text Feedback

Group A: **Agent** directly edits the user's code

Add a header on top



I changed the `index.html` file to use `<h1>` tags:

```
1 <div id="content">
+ <h1>Zic-Zac-Zoe</h1>
2 <div id="board">
3 ...
4 </div>
5 </div>
```



Group B: **Chatbot** w/ high-level syntax – users write code

Add a header on top



You can add this using `<h1>` tags:

```
<h1>title</h1>
```

```
1 <div id="content">
+ <h1>Zic-Zac-Zoe</h1>
2 <div id="board">
3 ...
4 </div>
5 </div>
```



RQ1: Do users **comprehend** the code they create with AI agents?

What would happen if `id` changed?

```
1 <div id="content">...</div>
```

(A) The board would disappear

...

(A) The board would no longer update



Which code snippets is yours?

```
* <div id="board">
2 ...
3 </div>
```

Left is Mine



```
* <div id="grid">
2 ...
3 </div>
```

Right is Mine

RQ2: Can users **extend** their initial code without AI agents?

How can I add a reset button?



Here's the syntax for a `<button>`:

```
<button>name</button>
```



```
1 <div class="content">
2 <h1>Zic-Zac-Zoe</h1>
3 <div id="board">
4 ...
5 </div>
6 </div>
+ <button id="reset">
+ Reset</but I
```



Step 3: Extend your code without agents (understanding metric 2)

Study Design on Code Understanding

Background Quiz (\$2.1) ▶ Warm-Up ▶ Initial Website Task (\$2.2) ▶ Warm-Up ▶ Comprehension Test (\$2.3) ▶ Extension Task (\$2.4) ▶ Text Feedback

Group A: **Agent** directly edits the user's code



Group B: **Chatbot** w/ high-level syntax – users write code



RQ1: Do users **comprehend** the code they create with AI agents?

What would happen if `id` changed?

```
1 <div id="content">...</div>
```

(A) The board would disappear

...

(A) The board would no longer update



Which code snippets is yours?

```
* <div id="board">
2 ...
3 </div>
```

Left is Mine



```
* <div id="grid">
2 ...
3 </div>
```

Right is Mine

RQ2: Can users **extend** their initial code without AI agents?

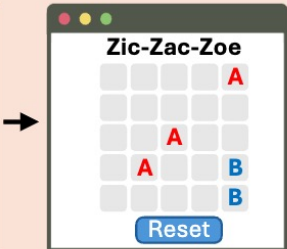
How can I add a reset button?

Here's the syntax for a `<button>`:

```
<button>name</button>
```



```
1 <div class="content">
2 <h1>Zic-Zac-Zoe</h1>
3 <div id="board">
4 ...
5 </div>
6 </div>
+ <button id="reset">
+ Reset</but I
```

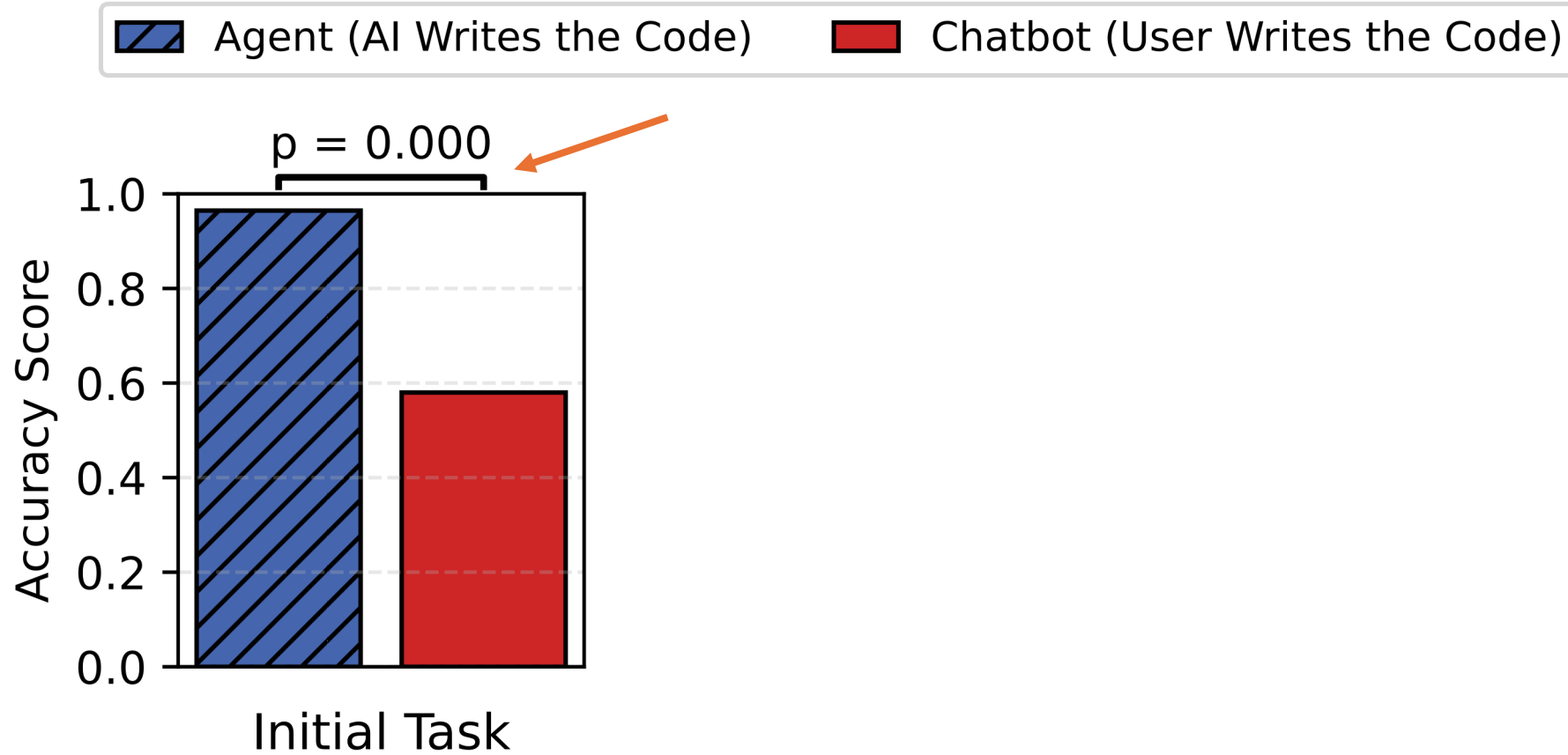


We define three more requirements that users must pass
But now users only have the chatbot — surprise!

Overall: Users don't understand their code w/ agents! (n=54 students)

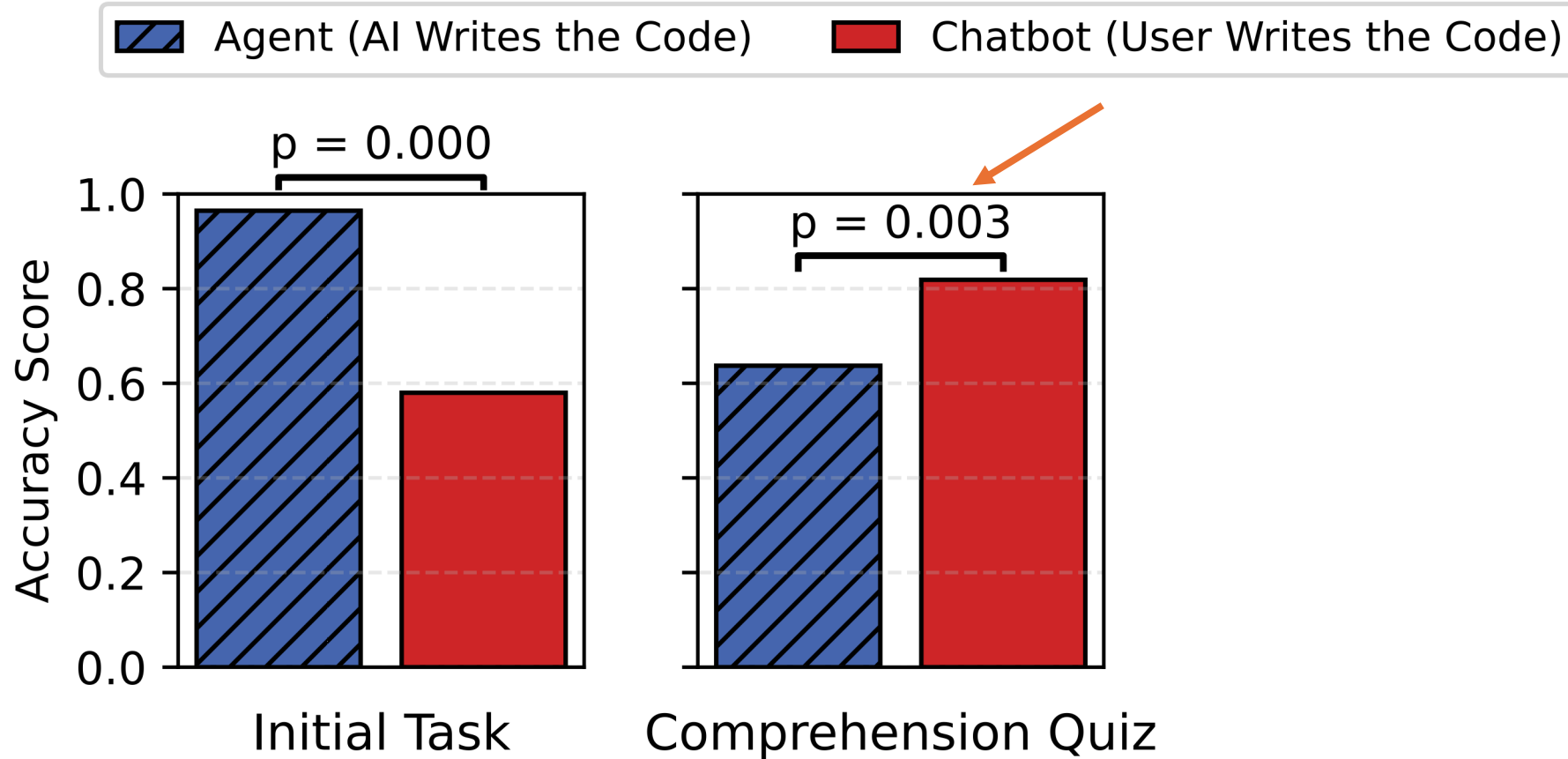


Overall: Users don't understand their code w/ agents! (n=54 students)



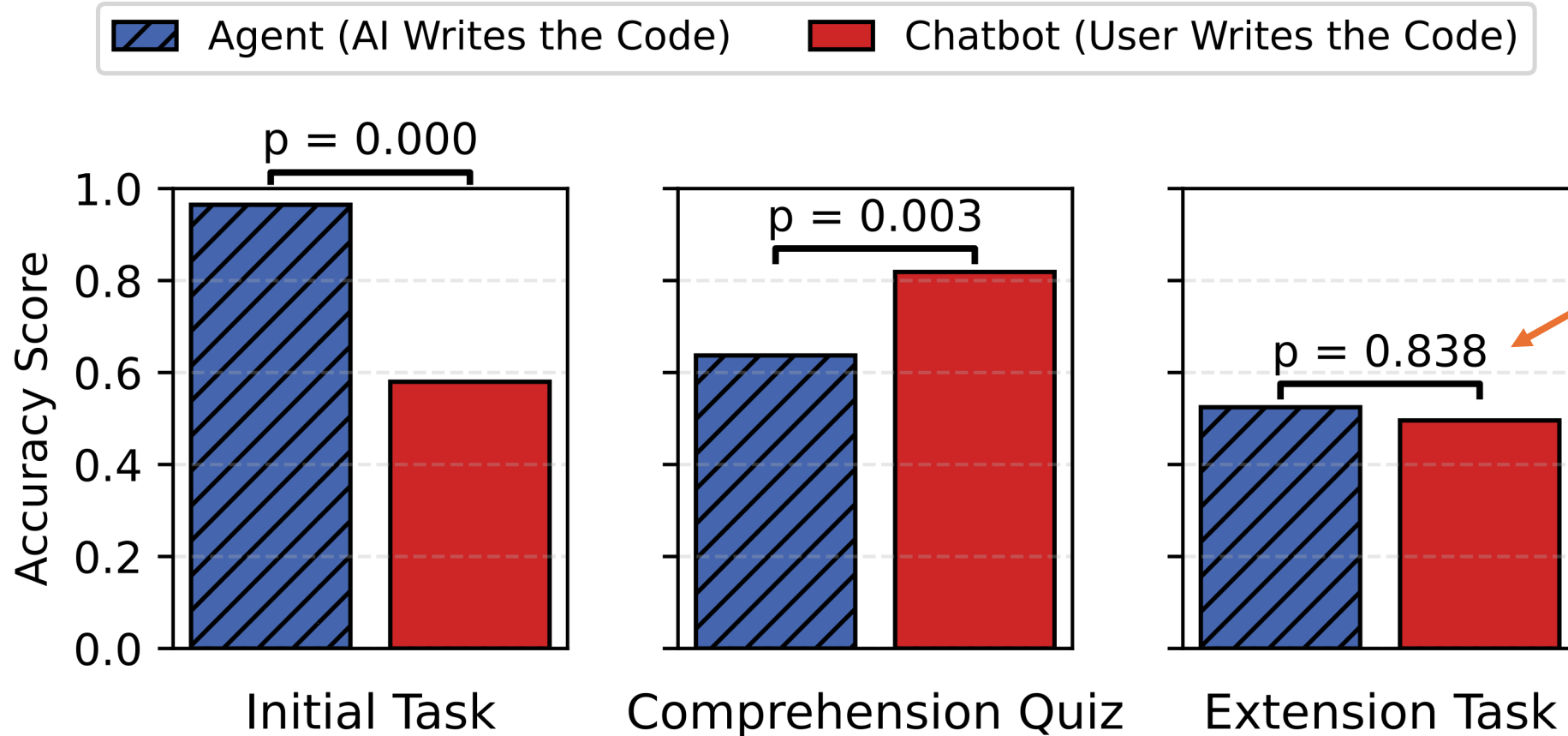
Users are more productive with agents (initial task)

Overall: Users don't understand their code w/ agents! (n=54 students)



Users are more productive with agents (initial task), but their comprehension is worse!

Overall: Users don't understand their code w/ agents! (n=54 students)



Users are more productive with agents (initial task), but their comprehension is worse!

When agents are gone, gains from agents in the initial task do not transfer to the extension task

What might we do to avoid poor comprehension with agents?

1) Avoid “lazy” prompting strategies — write prompts in technical terms 🧐👉

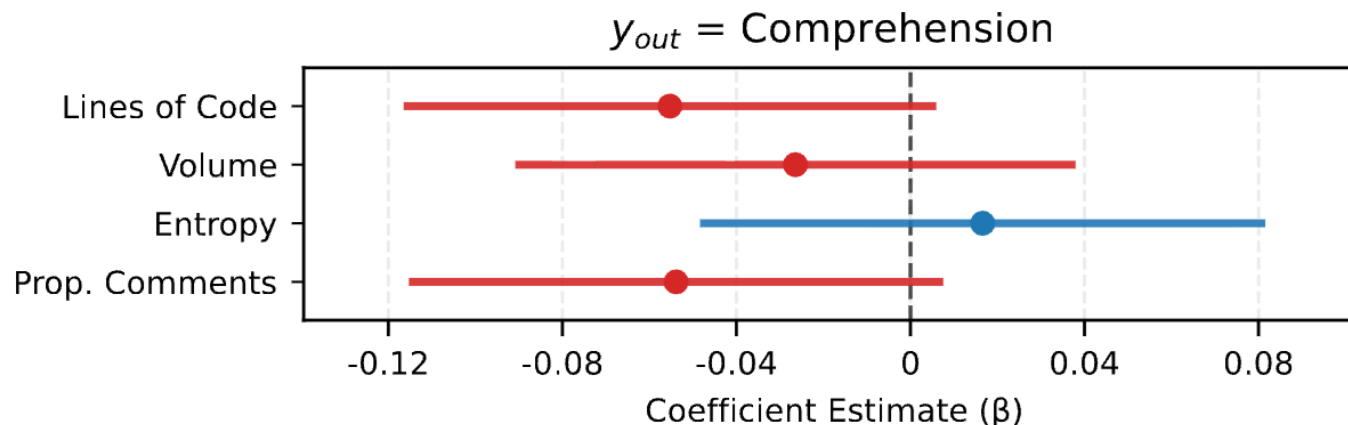
Prompting Strategy

Prompting Strategy	Comp. (↑)
Asking the agent to explain the codebase (“ <i>First, explain the whole project</i> ”)	0.635
Copying criteria (“ <i>Add a status element in a paragraph tag...</i> ”)	0.654
Iterative debugging (“ <i>Now there is a problem where it only shows player B...</i> ”)	0.692
Turning criteria into syntax (“ <i>Broadcast <u>statusMessage</u> to a <u>p</u> tag under the <u>h1</u> tag</i> ”)	0.743

2) Actually read your code 🧠

Interaction	Comp. (↑)
Auto-Accept	0.615
Override Changes	0.661
Hit Accept (All)	0.664
Hit Accept (Each)	0.777

3) Strive for simpler code — less lines, comments 🧘



And users acknowledge: code understanding w/ agents is cooked

From 1–5, how much do you agree:	Agent	Chatbot
----------------------------------	-------	---------

Users say coding agents are:

And users acknowledge: code understanding w/ agents is cooked

From 1–5, how much do you agree:	Agent	Chatbot
The agent/chatbot was helpful	4.7 ± 0.2	3.3 ± 0.2
The agent/chatbot took mental effort	1.9 ± 0.2	3.7 ± 0.2

Users say coding agents are:

➤ easier / more helpful

And users acknowledge: code understanding w/ agents is cooked

From 1–5, how much do you agree:	Agent	Chatbot
The agent/chatbot was helpful	4.7 ± 0.2	3.3 ± 0.2
The agent/chatbot took mental effort	1.9 ± 0.2	3.7 ± 0.2
I reviewed agent/chatbot responses	3.2 ± 0.3	4.0 ± 0.2
The code I submitted feels like my own	2.5 ± 0.2	3.7 ± 0.2
I understand how my code works	3.3 ± 0.3	4.2 ± 0.2
I could easily extend my code	3.1 ± 0.3	3.2 ± 0.2

Users say coding agents are:

- easier / more helpful
- harder to use for understanding

And users acknowledge: code understanding w/ agents is cooked

From 1–5, how much do you agree:	Agent	Chatbot
The agent/chatbot was helpful	4.7 ± 0.2	3.3 ± 0.2
The agent/chatbot took mental effort	1.9 ± 0.2	3.7 ± 0.2
I reviewed agent/chatbot responses	3.2 ± 0.3	4.0 ± 0.2
The code I submitted feels like my own	2.5 ± 0.2	3.7 ± 0.2
I understand how my code works	3.3 ± 0.3	4.2 ± 0.2
I could easily extend my code	3.1 ± 0.3	3.2 ± 0.2
I prefer no agent/chatbot in new tasks	1.4 ± 0.2	1.9 ± 0.2
I prefer the chatbot in new tasks	3.2 ± 0.3	2.7 ± 0.2
I prefer the agent in new tasks	4.2 ± 0.2	3.9 ± 0.2
I prefer switching between agent/chatbot	4.6 ± 0.2	4.1 ± 0.3

Users say coding agents are:

- easier / more helpful
- harder to use for understanding
- what they would still use in general

Users know agents harm understanding, but we still use them :)

And users acknowledge: code understanding w/ agents is cooked

From 1–5, how much do you agree:	Agent	Chatbot
The agent/chatbot was helpful	4.7 ± 0.2	3.3 ± 0.2
The agent/chatbot took mental effort	1.9 ± 0.2	3.7 ± 0.2
I reviewed agent/chatbot responses	3.2 ± 0.3	4.0 ± 0.2
The code I submitted feels like my own	2.5 ± 0.2	3.7 ± 0.2
I understand how my code works	3.3 ± 0.3	4.2 ± 0.2
I could easily extend my code	3.1 ± 0.3	3.2 ± 0.2
I prefer no agent/chatbot in new tasks	1.4 ± 0.2	1.9 ± 0.2
I prefer the chatbot in new tasks	3.2 ± 0.3	2.7 ± 0.2
I prefer the agent in new tasks	4.2 ± 0.2	3.9 ± 0.2
I prefer switching between agent/chatbot	4.6 ± 0.2	4.1 ± 0.3

Some salient themes + quotes:

1) Lack of thinking:

“I didn’t have to think at all.”

2) Agents are hard to control

“My agent would sometimes change somewhere unexpected or even destroy the original functions”

3) Students don’t want to learn...

“What’s the point of learning all this when the AI can build it out for you?”

Takeaways!



Interventions beyond prompting + reviewing for understanding



Using personalized assessments to evaluate difficult constructs

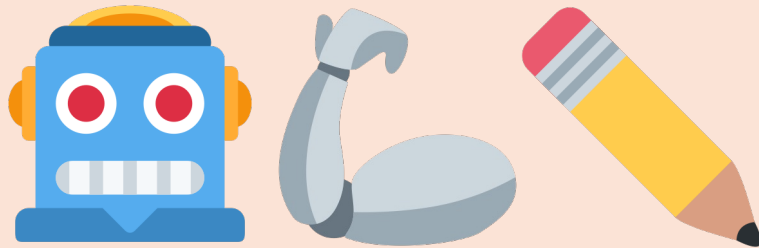


Changing how we reward (e.g., hire) coding to account for agents

While agents are getting better + better at editing code...

Understanding is still a bottleneck 🙅

RQ1: Which abilities beyond editing do coding agents need, but lack?



[Programming is Not Just Editing! The Importance of Code Understanding for Agents \(Under Review\)](#)

RQ2: Does optimizing agents for editing support human developers?



[\(Im\)Paired Programming: Coding Agents Improve Productivity but Harm Understanding \(Under Review\)](#)

A mix of NLP + HCI efforts will be needed to make *true* progress!